

We use some essential cookies to make our website work.

We use optional cookies, as detailed in our [cookie policy](#), to remember your settings and understand how you use our website.

[Accept optional cookies](#)

[Reject optional cookies](#)

Documentation

or [Ask](#)

Computers

[Getting started](#)

[Raspberry Pi OS](#)

Configuration

[Configuration methods](#)

[Audio and visual output](#)

[Networking](#)

[Wireless connection](#)

[System and localisation](#)

[User access and management](#)

[Remote access](#)

[Hardware communication](#)

[Performance](#)

[LED behaviour](#)

[Boot behaviour](#)

[Firmware](#)

[External storage](#)

[Operational security](#)

[Further reference](#)

[config.txt](#)

[The Linux kernel](#)

[Remote access](#)

[Camera software](#)

[AI software](#)

[Raspberry Pi computer hardware](#)

[Keyboard computers](#)

[Compute Module hardware](#)

[Processors](#)

[I/O controllers](#)

[Software sources](#)

[Legacy config.txt](#)

Configuration

Key Raspberry Pi settings are configured as part of the setup process using Raspberry Pi Imager, described in [Install using Imager](#). After installation, you can change your settings and configure additional settings; you can do this in different ways depending on your use case and technical level, described in [Configuration methods](#).

Configuration methods

[Edit this on GitHub](#)

There are several ways to configure your Raspberry Pi settings. While each method uses a different interface, they typically function by modifying `/boot/firmware/config.txt` and other relevant configuration files. The three main configuration methods are:

- [Desktop graphical user interface \(GUI\)](#)
- [Interactive raspi-config text user interface \(TUI\)](#)
- [Direct command-line interface \(CLI\)](#)

If your system includes a desktop environment, you can configure common settings from the desktop GUI. For more granular control, or if you're using Raspberry Pi OS Lite or have a [headless setup](#), Raspberry Pi OS includes the interactive `raspi-config` TUI, which is a Terminal-based configuration tool for managing common system settings.

The desktop GUI is the most user-friendly, but the method you choose depends on a number of factors, like whether you're using a headless computer (a Raspberry Pi with no output display connected) and your comfort level with the command line. For example, if you're using a headless Raspberry Pi with no connected display, you can't access a graphical desktop environment. In this case, you must use the `raspi-config` configuration utility or manually edit the relevant configuration files from the command line.

Some other settings can only be configured from the command line or in console mode. Although these don't appear in the desktop GUI or in the `raspi-config` TUI, they're still described in the relevant categories on this page.

Desktop GUI

If your system includes a desktop environment, the desktop GUI provides a visual, interactive layer for Raspberry Pi OS. When connected to a display, this interface allows you to control your Raspberry Pi using icons, menus, windows, and a pointer.

Most settings are managed through the **Control Centre**, but some options are available elsewhere on your desktop. For example, you can configure Wi-Fi from the wireless icon in the top-right corner of the system tray on your Raspberry Pi desktop.

To open the Control Centre, select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.

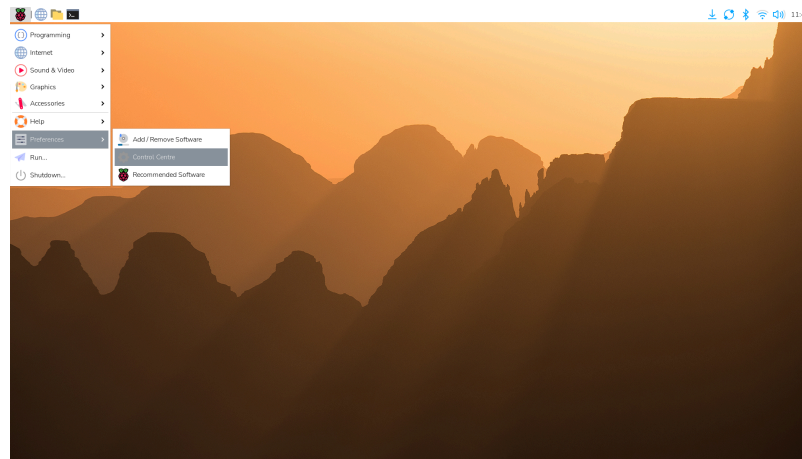
On this page

- [Configuration methods](#)
- [Audio and visual output](#)
- [Networking](#)
- [Wireless connection](#)
- [System and localisation](#)
- [User access and management](#)
- [Remote access](#)
- [Hardware communication](#)
- [Performance](#)
- [LED behaviour](#)
- [Boot behaviour](#)
- [Firmware](#)
- [External storage](#)
- [Operational security](#)
- [Further reference](#)

options

Accessories

Microcontrollers

**Figure 1.**

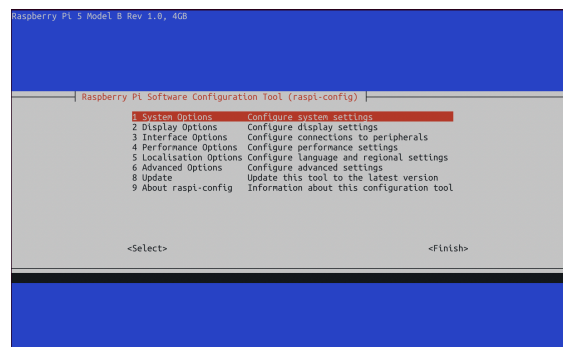
How to access the Control Centre from the Raspberry Pi desktop

NOTE

Instead of **Control Centre**, Raspberry Pi OS *Bookworm* uses a configuration tool called **Raspberry Pi Configuration**.

Interactive raspi-config TUI

Raspberry Pi OS includes `raspi-config`, which is a Terminal-based configuration tool for managing common system settings. It provides a structured menu-based interface that you navigate through using a keyboard.

**Figure 2.**

The main menu of the `raspi-config` text user interface (TUI)

You might use the `raspi-config` TUI if:

- You want or need more granular control than the desktop GUI.
- You're using Raspberry Pi OS Lite.
- You're running your Raspberry Pi in a headless environment.

To open the `raspi-config` tool, launch the Terminal on your Raspberry Pi device and then enter `sudo raspi-config`. Then:

- To navigate through the settings list, use the up and down arrow keys on your keyboard.
- To select a highlighted setting, press the **Enter** or **Return** key on your keyboard. You can also select a highlighted setting by using the right arrow or **Tab** key to highlight `<Select>` and then press the **Enter** or **Return** key on your keyboard.
- To return to the settings list, use the left arrow key on your keyboard or the **Tab** key on your keyboard.

- To jump ahead alphabetically, type the relevant letter. For example, type **E** to jump ahead to **Europe** in the timezone list.
- To close `raspi-config`, use the right arrow on your keyboard or the **Tab** key on your keyboard to select <Finish> and then press the **Enter** or **Return** key on your keyboard.

Direct CLI

You can manually configure some settings by editing system files or running commands in the Terminal or in console mode. Other more advanced settings, such as configuring a custom fullscreen splash image, can only be configured manually through the CLI.

Audio and visual output

Edit this on [GitHub](#)

Raspberry Pi OS supports multiple audio output and display modes. This section summarises how to configure audio and display settings on your Raspberry Pi, including how to:

- [Change audio output](#) from the desktop and from the command line.
- [Choose audio device profile](#) from the desktop and from the command line.
- [Configure display settings](#) (resolution, frequency, orientation, and scaling) from the desktop; if you're a console mode user, use the kernel command line instead.
- [Configure screen blanking](#) from the desktop, using the interactive `raspi-config` TUI, and from the command line.
- [Configure video output](#) (composite versus HDMI) using the interactive `raspi-config` TUI.
- [Configure onscreen keyboard](#) and choose what monitor it appears on from the desktop and using the interactive `raspi-config` TUI.

Change audio output

Raspberry Pi OS supports multiple audio output modes that you can choose from: **HDMI**, the **3.5 mm headphone jack** (if your device has one), and **USB audio devices**. By default, your Raspberry Pi outputs audio to the first HDMI device. If no HDMI device is detected, it automatically routes audio to the headphone jack or a connected USB device. You can change where audio is played from the desktop GUI or from the command line.

Desktop

CLI

The audio output selector on your Raspberry Pi desktop allows you to choose between multiple audio devices connected to your Raspberry Pi. On newer Raspberry Pi devices, this can include audio through one of the HDMI ports or USB ports.

If you've connected an audio device through the jack on the back of a Raspberry Pi monitor, it still appears as HDMI in the list because the monitor is attached to the HDMI port on your Raspberry Pi.

To change the audio output destination through the desktop:

1. Right-click the volume icon in the top-right corner of the system tray on your desktop. This opens the audio output selector.
2. Select the audio output of the device that you want to use.

Choose audio device profile

A device profile defines how an audio device is configured and what capabilities it exposes. Different profiles are optimised for different use cases. You can choose the

device profile from the desktop and, on newer Raspberry Pi devices, from the command line.

Desktop

CLI

1. Right-click the volume icon in the top-right corner of the system tray on your desktop.
2. Select **Device Profiles**.
3. Choose from one of the following options:

◦ **Off**. Turns off audio output for the Raspberry Pi.

◦ **Digital Stereo**. Outputs standard two-channel (left and right) audio. This is the default profile and works with most speakers, headphones, and monitors.

◦ **Pro Audio**. Exposes the maximum number of channels across every audio device, allowing greater control over signal routing. We don't recommend choosing this device profile unless you require fine-tuned control over audio output. For more information about the Pro Audio profile, see [PipeWire's FAQ](#).

Configure display settings

With most HDMI monitors, Raspberry Pi OS uses the highest resolution and refresh rate supported by your Raspberry Pi and your monitor. Connect the cables before turning on your Raspberry Pi.

Table 1.
Display capabilities of Raspberry Pi models

Models	HDMI type	Cable adapter	Max displays	Max resolution
Raspberry Pi Zero, Zero W, and Zero 2 W	Mini HDMI	Requires a mini-HDMI-to-HDMI cable or adapter	1	Depends on the display; usually up to 1920 x 1080 pixels
Raspberry Pi 4B and 400, and Compute Module 4	Dual micro HDMI	Requires a micro-HDMI-to-HDMI cable or adapter for each display you want to attach	2 (except CM4S)	Two 4K at 30 Hz or one 4K at 60 Hz
Raspberry Pi 5, 500, and 500+, and Compute Module 5	Dual micro HDMI	Requires a micro-HDMI-to-HDMI cable or adapter for each display you want to attach	2	Two 4K at 60 Hz

Raspberry Pi 4B, 400, and Compute Module 4, support either:

- Two displays with resolution up to 3840 × 2160 pixels (4K; Ultra HD) at a 30 Hz refresh rate, or resolution up to 1920 × 1080 pixels (Full HD) at a 60 Hz refresh rate.

• The display connected the HDMI0 port can support a resolution up to 3840 × 2160 pixels (4K; Ultra HD) at a 60 Hz refresh rate if you set the `hdmi_enable_4kp60=1` flag in `/boot/firmware/config.txt`; the second display still supports a resolution up to 3840 × 2160 pixels (4K; Ultra HD) at a 30 Hz refresh rate or 1920 × 1080 pixels (Full HD) at a 60 Hz refresh rate.

Raspberry Pi 5, 500, 500+, and Compute Module 5 support two displays with resolution up to 3840 × 2160 pixels (4K; Ultra HD) at a 60 Hz refresh rate with no additional configuration. You can adjust your display resolution and screen orientation through the desktop and through the console.

NOTE

Raspberry Pi OS Bookworm was the final version to support `raindrop` or `arandr` to configure display behaviours.

The command line instructions apply specifically to Raspberry Pi OS Lite or systems operating without a graphical desktop environment. If you're using a desktop GUI, follow the desktop instructions instead.

On systems without a desktop environment (running in console mode), display settings must be configured manually. Recent Raspberry Pi OS versions allow you to use Kernel Mode Setting (KMS) to configure display settings. This is done by adding the `video=` parameter to your kernel command line file (`cmdline.txt`). For more information about this file, see [Configure the kernel command line](#).

Desktop CLI (basic resolution) CLI (rotation and reflection)

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **Screens** tab in the left-side menu.
3. Select the screen that you want to configure by one of the following methods:
 - Right-click the box representing the screen that you want to configure.
 - Open the **Screens** dropdown menu and select the screen you want to configure.
4. Choose one of the following properties to configure:
 - **Resolution**. Choose a screen resolution (for example, 1920 × 1080).
 - **Frequency**. Choose a refresh rate (for example, 60 Hz).
 - **Orientation**. Choose between **Normal**, **Left**, **Inverted**, and **Right**.
 - **Scaling**. Choose between **1.0**, **1.5**, **2.0**, and **3.0** times the scale of the original screen.
5. Select **Apply** in the bottom-right corner of the **Control Centre**. Your changes are displayed for 10 seconds and you're asked to confirm that the changes are correct before continuing.
6. To confirm your changes, select **OK**. If you don't confirm the changes, the screen reverts back to its previous setting.

Configure screen blanking

You can configure your Raspberry Pi to blank the screen after a period of inactivity. When screen blanking is enabled, Raspberry Pi OS blanks the screen after ten minutes of inactivity by default.

You can configure screen blanking through the desktop GUI, using the `raspi-config` TUI, and from the command line.

Desktop `raspi-config` CLI

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **Display** tab in the left-side menu.
3. Use the toggle to turn **Screen Blanking** on or off.
4. Select **Close** in the bottom-right corner of the **Control Centre**.

Configure video output

For flagship models of Raspberry Pi, you can use the interactive `raspi-config` TUI to configure video output to be composite video or HDMI. Enabling composite video output disables HDMI output for all models except Raspberry Pi 5. To enable or disable composite video output using the interactive `raspi-config` TUI:

1. Open the Terminal and run `sudo raspi-config`.
2. Use the keyboard to navigate to and select **2 Display Options > D4 Composite**.
3. Select **Yes** or **No** in response to the question **Would you like composite video output to be enabled?**
4. Select **<OK>** to confirm this choice.
5. Select **<Finish>** when you're done.
6. Reboot your Raspberry Pi for the change to take effect.

Configure onscreen keyboard

You can configure your Raspberry Pi to show an onscreen keyboard and choose which screen this keyboard is displayed on through the desktop GUI and using the interactive `raspi-config` TUI. In the `raspi-config` TUI, the following settings are configured separately:

- Go to **D6 Onscreen Keyboard** to enable or disable the onscreen keyboard.
- Go to **D7 Keyboard** to specify which screen the onscreen keyboard is displayed on.

Desktop	raspi-config (enable or disable)	raspi-config (specify screen)
---------	----------------------------------	-------------------------------

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **Display** tab in the left-side menu.
3. Choose one of the following options from the **On-screen keyboard** dropdown menu:
 - **Enabled always**, which means the keyboard is always onscreen.
 - **Enabled if touchscreen found**, which means the keyboard is onscreen when a touchscreen is attached to your Raspberry Pi.
 - **Disabled**, which means the keyboard isn't on screen.
4. If you have more than one screen attached to your Raspberry Pi, choose a screen to display your keyboard on from the **On-Screen Keyboard Location** dropdown menu.
5. Select **Close** in the bottom-right corner of the **Control Centre**.

Networking

[Edit this on GitHub](#)

Raspberry Pi OS provides flexible options for configuring network settings. This section summarises how to manage and customise your Raspberry Pi's network configuration, including how to:

- **Change the hostname** of your Raspberry Pi from the desktop and by using the interactive `raspi-config` TUI.
- **Manage DHCP and static IP** using a manual method suitable for your use case.
- **Set network priority** from the desktop and from the command line.
- **Enable or disable predictable network interface names** using the interactive `raspi-config` TUI. This is an advanced option that is off by default.

- **Enable or disable link-local address fallback** using the interactive `raspi-config` TUI. This is an advanced option that is off by default.
- **Configure your Raspberry Pi to use a proxy server** using the interactive `raspi-config` TUI. This is an advanced option that is off by default.

Change the hostname

Setting a hostname is useful for the following reasons:

- When you set a hostname (for example, `my-pi`), you can access your Raspberry Pi through **mDNS** rather than the longer, changeable IP address. For example, instead of `ssh pi@192.168.1.42`, you can use `ssh pi@my-pi.local`.
- If you have multiple Raspberry Pi devices on the same network, you can ensure you're always connecting to the right Raspberry Pi by using unique hostnames (for example, `my-pi`, `pi-hole`, and `retro-pi`).

Your hostname isn't case sensitive and may only contain:

- ASCII letters from **a** through to **z**.
- Digits from **0** through to **9**.
- Hyphens (-).

No other symbols, punctuation, or blank spaces are permitted.

Desktop

raspi-config

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **System** tab in the left-side menu.
3. Select **Change Hostname**.
4. Enter a new hostname in the **Enter new hostname** field.
5. Select **OK** in the bottom-right corner of the window.
6. Select **Close** in the bottom-right corner of the **Control Centre**.

Manage DHCP and static IP

In a typical home network, DHCP (Dynamic Host Configuration Protocol) is managed by your router. It automatically assigns IP addresses and other networking settings (like the gateway and DNS servers) to devices as they connect.

Using DHCP allows your Raspberry Pi to connect to a network (through Wi-Fi or Ethernet) without manually configuring an IP address and other networking settings.

By default, IP addresses are temporary and can change over time as your router reassigns them as needed. If you're running services on your Raspberry Pi (such as a web server or SSH access), it's recommended that the device always has the same IP address. There are two main ways to achieve this:

- **DHCP reservation (recommended)**. Sometimes called a static lease, a DHCP reservation is configured on your router. It ensures that your Raspberry Pi always receives the same IP address based on its MAC address and keeps all IP management centralised on the router. Setup steps vary depending on your router's manufacturer and interface.
- **Static IP on the device**. You can manually configure a static IP address directly on your Raspberry Pi using `nmccli`. This requires manually setting the IP address, subnet mask, gateway, and DNS. We don't recommend doing this because misconfiguration can lead to connectivity issues. You must ensure that the chosen IP is outside the router's DHCP pool and doesn't conflict with other devices.

Set network priority

When your Raspberry Pi detects multiple known networks, it can connect to any of them by default. You can prioritise networks to control which network your Raspberry Pi prefers. If you set network priority, your Raspberry Pi tries to connect to the in-range network with the highest non-negative priority:

- Networks with higher numbers have higher priority; your Raspberry Pi tries to connect to the largest positive number first.
- Networks set to **0** have neutral priority, which is the default. A network set to **0** is chosen after other networks with a higher positive priority number.
- Networks with a negative number are low priority; your Raspberry Pi only connects to a network with negative priority if no other known network is available.

For example, consider the following four networks:

- Snake (-1)
- Rabbit (0)
- Dog (1)
- Cat (100)

If all of the above networks are in range, your Raspberry Pi first attempts to connect to the **Cat** network. If this network fails, it then attempts to connect to the **Dog** network, then the **Rabbit** network. If your Raspberry P detects no other known networks, it attempts to connect to the **Snake** network.

Desktop

CLI

1. Select the wireless icon in the top-right system tray. This icon shows the strength of the Wi-Fi signal currently connected and opens **Network Manager**.
2. Select **Advanced Options > Edit Connections**. This opens the **Network Connections** window.
3. Find and select the network that you want to prioritise.
4. Select the blue gear icon at the bottom of the window.
5. Open the **General** tab.
6. Ensure that **Connect automatically with priority** is selected.
7. Set the numeric priority value. This controls which network your Raspberry Pi prefers when multiple known networks are available.
8. Select **Save** in the bottom-right corner of the window, then close the **Network Connections** window.

Enable or disable predictable network interface names

You can enable or disable more specific interface names to make network device naming consistent across reboots and hardware changes. This is an advanced user option that is off by default because it replaces the traditional `eth0` and `wlan0` naming schemes on some devices. Naming behaviour can vary between Raspberry Pi models and configurations.

1. Open the Terminal and run `sudo raspi-config`.
2. Use the keyboard to navigate to and select `6 Advanced Options > A2 Network Interface Names`.
3. Select `Yes` or `No` in response to the question **Would you like to enable predictable network interface names?**
4. Select `<OK>` to confirm this choice and then `<Finish>` when you're done.

5. Reboot your Raspberry Pi for the change to take effect.

Enable or disable link-local address fallback

You can enable or disable link-local address fallback to assign a temporary 169.254.x.x address. This address might be assigned even if a DHCP server is available: the system first assigns a link-local address, then replaces it when a DHCP lease is obtained. If no DHCP response is received, the link-local address remains in use.

This is an advanced user option that is off by default because it can interfere with network configuration and isn't typically needed on most networks where DHCP is available. Only enable this option if you understand this behaviour.

1. Open the Terminal and run `sudo raspi-config`.
2. Use the keyboard to navigate to and select 6 Advanced Options > A14 Link-local Fallback.
3. Select Yes or No in response to the question **Would you like to enable link-local fallback?**
4. Select <OK> to confirm this choice and then <Finish> when you're done.
5. Reboot your Raspberry Pi for the change to take effect.

Configure your Raspberry Pi to use a proxy server

A proxy server acts as an intermediary between a client device and the internet. To configure your Raspberry Pi to use a proxy server, you need:

- The IP address or hostname and port of your proxy server.
- A username and password for your proxy server (if required).

This is an advanced configuration that you should only configure if your network requires a proxy (for example, in managed, corporate, restricted environments).

1. Open the Terminal and run `sudo raspi-config`.
2. Use the keyboard to navigate to and select 6 Advanced Options > A3 Network Proxy Settings.
3. Configure environment variables for proxy usage. Choose one or more of the following options depending on your needs. Each option must be configured separately, meaning you must start from step 1 for each option you want to configure:
 - P1 All applies the same proxy settings to all network traffic.
 - P2 HTTP sends all HTTP traffic through the proxy.
 - P3 HTTPS sends all HTTPS (secure) traffic through the proxy.
 - P4 FTP sends all FTP traffic through the proxy.
 - P5 Exceptions specifies addresses that should bypass the proxy.
4. Select <OK> and then <OK> again.
5. Select <Finish> when you're done.
6. Reboot your Raspberry Pi for the change to take effect.

For options P1 through to P4, enter the proxy in the following format, replacing <proxy_ip_address> with the IP address of your proxy and <proxy_port> with the port of your proxy:

```
http://<proxy_ip_address>:<proxy_port>
```

If your proxy requires a username and password, add them using the following format, replacing <username> and <password> with the username and password you use to authenticate to your proxy.

```
http://<username>:<password>@<proxy_ip_address>:<proxy_port>
```

For option P5, enter a comma-separated list of addresses that can bypass the proxy. Add additional domains or IP ranges as needed.

```
no_proxy="localhost,127.0.0.1"
```

Wireless connection

This section covers how to:

- Configure a Wi-Fi connection, specifically, how to:
 - **Enable wireless networking (set WLAN country)**, which is a prerequisite for connecting to a wireless network, which you can do from the desktop and using the interactive `raspi-config` TUI.
 - **Find networks** to connect to, which you can do from the desktop and from the command line.
 - **Connect to a wireless network**, which you can do from the desktop and from the command line.
 - **Enable or disable WLAN power saving**, which is an advanced option set through the interactive `raspi-config` TUI.
- **Connect Bluetooth peripherals**, such as a mouse or keyboard, which you can do from the desktop.

The Wi-Fi connection requires either a Raspberry Pi with built-in wireless connectivity or a wireless USB stick plugged into it.

Enable wireless networking (set WLAN country)

On Raspberry Pi devices equipped with dual-band wireless, networking is automatically disabled as a safety measure until a Wireless LAN (WLAN) country is set. This applies from Raspberry Pi 3B+ onwards, Compute Module 4 onwards, and all keyboard computers (Raspberry Pi 400, 500, and 500+).

Setting the WLAN country specifies where you use your Raspberry Pi. This allows your Raspberry Pi to choose the correct frequency bands for 5 GHz networking. When you've specified a WLAN country, you can use your Raspberry Pi device's built-in wireless networking module. You only need to set the WLAN country once unless you want to change it.

IMPORTANT

Selecting the correct Wi-Fi regulatory domain is critical because it determines available channels, transmit power, and overall network performance; incorrect settings can cause legal issues, increased interference, reduced capacity, and degraded connectivity.

Typically, WLAN country is set when you install an OS onto your boot media using Imager, but you can also set it up or change it later through the desktop GUI or from the Terminal (through the `raspi-config` TUI and using `nmcli` commands).

Desktop

raspi-config and nmcli

1. Select the Raspberry Pi icon in the top-left corner.
2. Select **Preferences > Control Centre**.
3. Open the **Localisation** tab.
4. Select **Set WLAN Country** at the bottom of the list.
5. Choose your country from the **Country** dropdown menu.
6. Select **OK** and then **Close**.

Find networks

Before connecting to a wireless network, you must find or specify an available one. On Raspberry Pi OS, you can view nearby Wi-Fi networks either using **Network Manager** in the desktop GUI, or through the command line.

Desktop

CLI

To view available Wi-Fi networks from the desktop, select the wireless icon in the top-right system tray. This icon shows the strength of the Wi-Fi signal currently connected and opens **Network Manager**. From Raspberry Pi OS **Bookworm** onwards, Network Manager is the default network configuration tool.

The Network Manager shows a list of available wireless networks. The icons next to each available network indicate:

- Whether the network is operating on a 5 GHz frequency band (**5G** icon).
- Whether the network is secured (padlock icon).
- The signal strength of the network (four-bar Wi-Fi icon).



Figure 3.
Wireless Manager menu

If a message appears telling you no access points (AP) were found, for example, 'No APs found - scanning', wait a few seconds for Network Manager to find your network.

If your wireless network isn't present in the menu because it has a hidden SSID, follow the instructions in [Connect to a wireless network](#).

Connect to a wireless network

You can set up wireless connection from the desktop and using `nmcli` (NetworkManager Command Line Interface) commands.

`nmcli` is a command-line method of configuring a system's network settings. The command line instructions outlined below use only a few `nmcli` commands. However, `nmcli` offers numerous commands that you might find useful for managing wireless connections. For more information, see the [nmcli network manager documentation](#).

NOTE

The following instructions work for WEP, WPA, WPA2, and WPA3 networks, but might not work for enterprise networks.

Desktop

Desktop (hidden network)

CLI

To change or add a wireless network from the desktop GUI, open the Network Manager, as described in [Find networks](#) and, if the wireless network you want to connect to isn't hidden:

1. Select the appropriate wireless network. If your wireless network isn't listed because it's a hidden network, follow the instructions for a hidden network instead.
 - If you don't already have a wireless network selected, or if you have previously turned it off, select **Turn On Wireless LAN**. Then, select a wireless network from the menu.
 - If you're already connected to a wireless network, choose a different one from the menu.
2. If prompted to, enter the password for that wireless network.

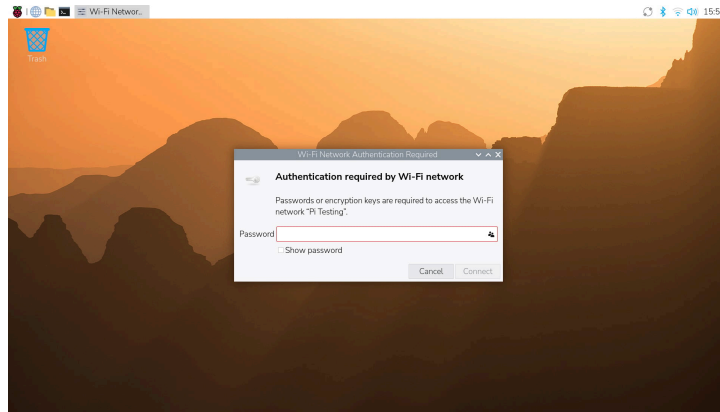


Figure 4.
Password prompt for connecting to a secure wireless network

3. Select **Connect** in the bottom-right corner of the window.

Enable or disable WLAN power saving

WLAN power saving is an advanced option in `raspi-config` that we don't recommend setting unless instructed to by a Raspberry Pi engineer. Disabling power saving is applied immediately and persists for future connections. Enable this setting with caution; it might improve connection stability but increase power consumption.

1. Open the Terminal and run `sudo raspi-config`.
2. Use the keyboard to navigate to and select `6 Advanced Options > A13 WLAN Power Save`.
3. Select **Yes** or **No** in response to the question **Would you like to enable WLAN power saving?**
4. Select `<OK>` to confirm this choice and then `<Finish>` when you're done.

Connect Bluetooth peripherals

If your Bluetooth device doesn't include a USB dongle, you can connect it from the desktop GUI. To discover and pair a Bluetooth keyboard or mouse:

1. From the top menu, select the Bluetooth icon directly to the left of the Wi-Fi icon.
2. Select **Add Device**. This opens the **Add New Device** window.
3. Wait for the device to appear in the window and then select it.
4. Select **Pair** in the bottom-right corner of the window.
5. Select **OK** to confirm that you'd like to pair with the device.

System and localisation

Edit this [on GitHub](#)

This section covers how to:

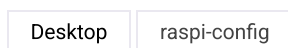
- **Change default browser.** Choose between Chromium or Firefox; the default browser if you don't choose one during installation is Chromium.
- Configure localisation settings, including how to:
 - **Configure locale settings**, which are the system-wide language and regional settings (country and character set).
 - **Configure time zone**, which is determined by the geographic area and city used to set the local time.
 - **Configure your keyboard** by specifying the physical model, language arrangement, and key variants of your keyboard.
- Configure advanced system settings, including how to:
 - **Expand the file system**, which might be useful if you've cloned your OS to a larger storage device and you're using an OS that doesn't auto-expand the filesystem to make use of the space.
 - **Configure logging**, which is used to set a storage location for logs.
 - **Enable or disable beta access**, which you can turn on if you're an opt-in tester user of Raspberry Pi OS.
 - **Switch between Wayland and X11**. We don't recommend switching to X11 because we are no longer developing for it.

The default browser and localisation settings can all be configured from the desktop and using the interactive `raspi-config` TUI. The advanced settings can only be configured using the interactive `raspi-config` TUI.

If you're looking for instructions for how to set the WLAN country on your Raspberry Pi, see **Enable wireless networking instead**. The instructions for setting WLAN country is in the **Wireless connection** section because it's a prerequisite for connecting to a wireless network.

Change default browser

You can change the default browser on your Raspberry Pi through the desktop and using the interactive `raspi-config` TUI. You can choose between **Chromium** and **Firefox**.



1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **System** tab in the left-side menu.
3. Choose between **Chromium** and **Firefox**.
4. Select **Close** in the bottom-right corner of the **Control Centre**.

Configure locale settings

Locales are a framework to switch between multiple languages and regions. They allow your Raspberry Pi to display your preferred language, date formats, currency symbols, and character sets. **UTF-8** locales are selected by default for new installations. **C**, **C.UTF-8**, and **POSIX** locales are also available, and you can also generate others to suit your needs.

When you configure your locale settings, you're configuring a system-wide default locale, which changes the language for all users of your Raspberry Pi.

Configuring locale from the desktop involves setting the following separately:

- Language, for example, **en** for English.
- Region (country), for example, **GB** for Great Britain.

- Character set (the character encoding standard for modern web and OS), for example, **UTF-8** for Unicode support.

Configuring locale using the `raspi-config` TUI involves setting the language, region, and character set with one string. For example, **en_GB.UTF-8** sets the locale to British English with UTF-8 character encoding.

Desktop raspi-config

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **Localisation** tab in the left-side menu.
3. Select **Set Locale** next to **Locale**.
4. Choose your language from the **Language** dropdown menu, for example, **en (English)**.
5. Choose your country from the **Country** dropdown menu, for example, **GB (United Kingdom)**.
6. Choose one of the available character sets from the **Character Set** dropdown menu, for example, **UTF-8**.
7. Select **OK** in the bottom-right corner of the **Locale** window.
8. Select **Close** in the bottom-right corner of the **Control Centre**.
9. Reboot your Raspberry Pi for the changes to take effect.

Configure time zone

Configuring your time zone involves first selecting a geographic region (for example, **Europe**) and then selecting the city that corresponds with your time zone based on where that city is located (for example, **London**). Selecting the correct time zone handles daylight saving time transitions automatically.

Desktop raspi-config

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **Localisation** tab in the left-side menu.
3. Select **Set Timezone** next to **Timezone**.
4. Choose the appropriate geographic area from the **Area** dropdown menu. For example, if you live in the UK, select **Europe**.
5. Choose the appropriate city that corresponds to your time zone. For example, if you live in the UK, select **London**.
6. Select **OK** in the bottom-right corner of the **Timezone** window.
7. Select **Close** in the bottom-right corner of the **Control Centre**.

Configure your keyboard

You can customise and configure your keyboard by changing any of the following settings:

- **Model**. The physical hardware design and number of keys on your device. For example, choose **Generic 105-key PC** if you have a standard full-sized keyboard with numeric keypad.
- **Layout**. The primary language and symbol arrangement assigned to the keys on your keyboard. For example, choose **English (UK)** for a layout where the " and @ symbols are swapped compared to a US keyboard.

- You can also choose an **Alternate Layout**, available only through the desktop Control Centre. This is an alternative keyboard layout that you can switch to using a toggle key that you specify. For example, you can choose **English (US)** as your alternative layout.
- If you choose an alternative layout, set a **Toggle Key** to switch between the **Layout** and any **Alternate Layout** you've set. This configuration is only available through the desktop Control Centre, and involves specifying a key or key combination such as **Ctrl + Alt**.
- Optionally, set an **Indicator** (a keyboard LED) for when an **Alternate Layout** is active, available only through the desktop Control Centre. Typically, this is either **Caps**, **Num**, or **Scroll**.
- **Variant**. Available only through the desktop Control Centre.
 - Choose specific modification of a layout for specialised hardware or regional preferences. For example, choose **English (UK, Macintosh)** for Apple keyboards where the **Alt** and **Cmd** keys are remapped. If you don't have a specific modification, the variant is the same as the layout.
 - You can also choose an **Alternative Variant**, which is a specific modification of a layout for specialised hardware or regional preferences. For example, choose **Cherokee** for Cherokee English.
- **AltGr**. Available only through `sudo raspi-config` in the Terminal. A modifier key used to input some characters that are unusual for the language of the keyboard layout, such as foreign currency symbols and accented letters.

Because the desktop **Control Centre** and `sudo raspi-config` each have their own configuration settings, you might have to follow both procedures to complete your keyboard customisation.

Desktop

raspi-config

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **Localisation** tab in the left-side menu.
3. Select **Set Keyboard** next to **Keyboard**.
4. From the **Model** dropdown menu, choose the appropriate keyboard model.
5. From the **Layout** dropdown menu, choose the appropriate keyboard layout.
6. From the **Variant** dropdown menu, choose the appropriate keyboard variant.
7. Optionally, select **Add alternate keyboard**. This reveals four more dropdown menus.
 - a. From the **Alternate Layout** dropdown menu, choose the layout of your alternate keyboard.
 - b. From the **Alternate Variant** dropdown menu, choose the variant of your alternate keyboard.
 - c. From the **Toggle Key** menu, choose which key or combination of keys allows you to switch between keyboard configurations, for example, **Left Alt + Caps**.
 - d. If you have a keyboard with indicator lights, from the **Indicator** dropdown menu, choose which LED on your keyboard lights up when the alternate keyboard is active: **None**, **Caps**, **Num**, or **Scroll**.

NOTE

Don't set **Num** as the indicator LED on the Raspberry Pi standalone USB keyboard or the Raspberry Pi 400. Doing so results in the keyboard generating letters instead of numbers on the right-side of the keyboard.

8. Select **OK** in the bottom-right corner of the **Keyboard** window.
9. Select **Close** in the bottom-right corner of the **Control Centre**.
10. Reboot your Raspberry Pi for the changes to take effect.

Expand the file system

Expanding your file system is an advanced option, typically only needed if you're moving to a larger boot media and you're not using Raspberry Pi OS. Expanding the OS partition allows your Raspberry Pi to use the full capacity of the new boot media.

In most cases, Raspberry Pi OS performs the expansion automatically on first boot.

WARNING

There is no confirmation step. When you choose to expand your file system, partition expansion starts immediately.

To expand the file system using the interactive `raspi-config` TUI:

1. Open the Terminal and run `sudo raspi-config`.
2. Use the keyboard to navigate to and select `6 Advanced Options > A1 Expand Filesystem`.
3. `raspi-config` automatically resizes the root partition and then displays a confirmation that the file system will be expanded on the next reboot.
4. Select `<OK>` and then `<Finish>` when you're done.
5. Reboot your Raspberry Pi for the change to take effect.

Configure logging

Logging is an advanced option, typically only needed if you need to change how system log data is stored or managed, for example, to reduce disk writes, retain logs across reboots, or disable logging for performance reasons.

To configure logging using the interactive `raspi-config` TUI:

1. Open the Terminal and run `sudo raspi-config`.
2. Use the keyboard to navigate to and select `6 Advanced Options > A12 Logging`.
3. Choose between one of the following options:
 - `1 Default`, which uses the system's default logging configuration.
 - `2 Volatile`, which stores logs in RAM only, typically under `/run/log/journal`. Logs are temporary and cleared on reboot.
 - `3 Persistent`, which stores logs on disk, typically under `/var/log/journal`, meaning logs survive reboots if you want to use them for troubleshooting over time. This mode also supports separate journal files for each user (log splitting).
 - `4 Auto`, which automatically determines the appropriate logging behaviour based on system configuration and available storage. Persistent logging is used if `/var/log/journal` exists; volatile logging is used if `/var/log/journal` doesn't exist.
 - `5 None`, which turns off system logging.
4. Select `<OK>` and then `<OK>` again.
5. Select `<Finish>` when you're done.

Enable or disable beta access

Beta access allows you to use newer, pre-release software packages and bootloader updates that are still being tested. This option is intended for advanced users who want to try the latest features and help identify issues.

NOTE

This is different from using `rpi-update`, which provides untested firmware updates and should only be used when specifically advised. Beta access is a safer, opt-in way to test newer software from a dedicated repository.

To enable or disable beta access using the interactive `raspi-config` TUI:

1. Open the Terminal and run `sudo raspi-config`.
2. Use the keyboard to navigate to and select `6 Advanced Options > A6 Beta Access`.
3. Choose between one of the following options:
 - `B1 Beta Software`, which enables the beta repository and allows installation of newer, pre-release packages and the latest bootloader updates.
 - `B2 Release Software`, which uses the standard, stable software repository recommended for most users.
4. Select `<OK>` and then `<OK>` again.
5. In a new Terminal, use `sudo apt update` to update your package list to apply the change.

Switch between Wayland and X11

From Raspberry Pi OS Bookworm onwards, all Raspberry Pi models run Wayland using `labwc` by default.

WARNING

We don't recommend switching from Wayland to X because we aren't actively developing X11.

To switch between Wayland and X11 using the interactive `raspi-config` TUI:

1. Open the Terminal and run `sudo raspi-config`.
2. Use the keyboard to navigate to and select `6 Advanced Options > A7 Wayland`.
3. Choose between one of the following options:
 - `W1 X11` (not recommended).
 - `W2 Labwc` (Wayland, which is recommended).
4. Select `<OK>` and then `<OK>` again.
5. Select `<Finish>` when you're done.

User access and management

Edit this [on GitHub](#)

This section covers how to manage users and passwords on your Raspberry Pi, including how to:

- **View existing users** from the command line to see which users exist on your Raspberry Pi device.
- **Add a user** from the command line to create a new user with a separate account and home directory on your Raspberry Pi.
- **Verify a user exists** from the command line to check that a new user exists and has a home directory.
- **Grant user access permissions** from the command line to give user accounts on your Raspberry Pi access to groups and superuser permissions.
- **Switch user** from the command line to change the active user on your Raspberry Pi.
- **Check permissions** from the command line to check the groups a user belongs to.
- **Delete a user** from your Raspberry Pi from the command line.

- **Change your password** from the desktop, in the TUI, or from the command line.
- **Require a password from startup** by turning off auto login from the desktop or in the TUI.
- **Turn off password for superuser commands** from the desktop or in the TUI.

Quick reference

This is a short list of some of the commands referenced by the procedures in this section.

Function	Command
List all existing users.	\$ ls -l /home
Check the group permissions for the current user.	\$ groups
Add a new user.	\$ sudo adduser
Switch to a different user.	\$ sudo su - <username>
Delete a user and their home directory.	\$ sudo deluser --remove-home <username>
Change the current user's password.	\$ passwd
Open the configuration centre.	\$ sudo raspi-config

View existing users

To get a list of existing users and their home directories on your Raspberry Pi device, run the following command:

```
$ ls -l /home
```

You can navigate to the home directory of a user listed here using the following command, replacing <username> with the username of the user:

```
$ cd /home/<username>  
$ ls -la
```

Add a user

To add a new user to your Raspberry Pi:

1. Enter the following command, replacing <username> with the username for the new user:

```
$ sudo adduser <username>
```
2. When prompted, enter a password for the new user.
3. Optionally, add additional information about the new user, such as their full name, room number, and so on. If you don't want to include this information, press **Enter** or **Return** on your keyboard to skip.
4. Press **Y** followed by **Enter** or **Return** to indicate that the information is correct.

Verify a user exists

When you create a new user, their home directory is automatically created at /home/<username>. To check the home directory for the new user, run the following command: /home/<username>/, replacing <username> with the user's username. This returns something like:

```
-bash: /home/<username>: Is a directory
```

Grant user access permissions

You can grant a user specific permissions by adding them to system groups.

Run the following command, replacing `<username>` with the new user's username. Including the `sudo` group in the following command gives the user superuser privileges.

```
$ sudo usermod -a -G adm,dialout,cdrom,sudo,audio,video,plugdev,games,users,
input,render,netdev,lpadmin,gpio,i2c,spi <username>
```

To check that the changes were applied successfully, [switch to the user account](#) and then [check their permissions](#), described below.

Switch user

To switch to a different user, run the following command, replacing `<username>` with the username of the user:

```
$ sudo su - <username>
```

Check permissions

To check the groups a user belongs to, [switch to the user account](#) then run the following command:

```
$ groups
```

Delete a user

To delete a user from your Raspberry Pi, enter the following command, replacing `<username>` with the username for the user. This deletes both the user and their home directory. To preserve the user's home directory, run the command without the `--remove-home` option.

```
$ sudo deluser --remove-home <username>
```

Press **Y** followed by **Enter** or **Return** on your keyboard to indicate that the information is correct.

Change your password

You can change password for the current user from the desktop, the TUI, or the command line.

Desktop	raspi-config	CLI
---------	--------------	-----

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **System** tab in the left-side menu.
3. Select **Change Password**.
4. In the **Enter new password** field, enter a new password.
5. In the **Confirm new password** field, enter the same password again.
6. Select **Close** in the bottom-right corner of the **Control Centre**.
7. Reboot your Raspberry Pi to apply your new password.

Require a password from startup

You can require users to enter their password when booting the Raspberry Pi by turning off **Auto Login**.

When you turn off auto login, you choose whether to separately require the password for signing in to the **console** and the **desktop**.

[Desktop](#)[raspi-config](#)

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **System** tab in the left-side menu.
3. To configure your Raspberry Pi to require a password when you boot into the console, turn off **Console Auto Login** using the respective toggle.
4. To configure your Raspberry Pi to require a password when you boot into the desktop, turn off **Desktop Auto Login** using the respective toggle.
5. Select **Close** in the bottom-right corner of the **Control Centre**.
6. Reboot your Raspberry Pi to apply your changes.

Turn off password for superuser commands

Prefixing a command in Raspberry Pi with `sudo` runs that command with superuser (admin) privileges. By default, Raspberry Pi OS requires you to enter your password when running `sudo` commands. This prevents unauthorised users from running `sudo` commands on your device.

You can override this setting after installation through either the desktop or by using the interactive `raspi-config` TUI.

[Desktop](#)[raspi-config](#)

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **System** tab in the left-side menu.
3. Use the toggle next to **Admin Password** to turn off the password requirement for `sudo` commands.
4. Select **Close** in the bottom-right corner of the **Control Centre**.
5. Reboot your Raspberry Pi to apply your changes.

Remote access

[Edit this on GitHub](#)

This section covers how to control your Raspberry Pi from another computer without being physically next to it. Specifically, this section covers how to:

- [Enable or disable SSH](#) for remote access to your Raspberry Pi command line. You can do this from the desktop, using the interactive `raspi-config` TUI, and manually from the command line.
- [Enable or disable Connect](#), a Raspberry Pi service that provides remote access to your Raspberry Pi desktop and Terminal with no manual network configuration. You can enable this from the desktop, using the interactive `raspi-config` TUI, and manually from the command line.
- [Enable or disable VNC](#) (virtual network computing) for remote access to your Raspberry Pi's desktop. You can do this from the desktop and using the interactive `raspi-config` TUI.

Enable or disable SSH

Enabling SSH allows you to remotely access the command line of your Raspberry Pi from another computer. You can enable SSH from the desktop, using the interactive `raspi-config` TUI, and manually from the command line. You can also enable SSH as part of the installation process using Imager. For more information, see [Install using Imager](#).

After you've enabled SSH, the next step is to connect to your Raspberry Pi from another computer using an SSH client. For more information, see the [Remote access](#) documentation.

[Desktop](#)[raspi-config](#)[CLI](#)

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **Interfaces** tab in the left-side menu.
3. Use the toggle next to **SSH** to turn it on or off.
4. Select **Close** in the bottom-right corner of the **Control Centre**.
5. Reboot your Raspberry Pi to apply your changes.

Enable or disable Connect

Raspberry Pi Connect allows you to remotely access the desktop of your Raspberry Pi from another computer. You can enable Raspberry Pi Connect from the desktop, using the interactive `raspi-config` TUI, and manually from the command line.

After you've enabled Raspberry Pi Connect, the next step is to link a Raspberry Pi device. For more information, see the [Remote access](#) and [Raspberry Pi Connect](#) documentation.

[Desktop](#)[raspi-config](#)[CLI](#)

Raspberry Pi connect is enabled and disabled from the Raspberry Pi Connect icon in the system tray in the top right of your desktop GUI. The Raspberry Pi Connect icon is two small squares connected by a circle.

- To enable Raspberry Pi Connect from the desktop, select the Raspberry Pi Connect icon in the top right of the system tray and then select **Turn On Raspberry Pi Connect**.
- To disable Raspberry Pi Connect from the desktop, select the Raspberry Pi Connect icon in the top right of the system tray and then select **Turn off Raspberry Pi Connect**.

Enable or disable VNC

Enabling VNC allows you to access your Raspberry Pi's desktop from another computer. You can enable VNC from the desktop, using the interactive `raspi-config` TUI.

After you've enabled VNC, the next step is to connect to your Raspberry Pi from another computer using a VNC client. For more information, see the [Remote access](#) documentation.

[Desktop](#)[raspi-config](#)

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **Interfaces** tab in the left-side menu.
3. Use the toggle next to **VNC** to turn it on or off.
4. Select **Close** in the bottom-right corner of the **Control Centre**.

Hardware communication

This section covers how to enable hardware communication protocols on your Raspberry Pi so that your Raspberry Pi can interact with external devices, such as

sensors, displays, and microcontrollers. Specifically, this section covers how to:

- **Enable or disable SPI** for high-speed communication with peripherals, such as small displays. You can do this from the desktop and using the interactive `raspi-config` TUI.
- **Enable or disable I2C** for communication with low-speed devices like sensors, RTC modules, and some HATs. You can do this from the desktop and using the interactive `raspi-config` TUI.
- **Enable or disable the one-wire interface** for simple communication with devices like temperature sensors and LEDs. You can do this from the desktop and using the interactive `raspi-config` TUI.
- **Enable or disable serial port** for UART communication with other devices. You can do this from the desktop and using the interactive `raspi-config` TUI.
- **Configure UARTs** for different purposes, such as selecting which UART is used for communication and adjusting serial settings. Most of these instructions involve using the command line.

Enable or disable SPI

Enabling SPI (Serial Peripheral Interface) allows your Raspberry Pi to communicate with compatible devices using a high-speed synchronous protocol. Specifically, enabling SPI does two things:

- Turns on the SPI interface on your Raspberry Pi.
- Loads the SPI kernel module at boot so that connected devices can communicate.

You can enable SPI from the desktop and using the interactive `raspi-config` TUI.

Desktop	raspi-config
---------	--------------

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **Interfaces** tab in the left-side menu.
3. Use the toggle next to **SPI** to turn it on or off.
4. Select **Close** in the bottom-right corner of the **Control Centre**.

Enable or disable I2C

Enabling I2C (Inter-Integrated Circuit) allows your Raspberry Pi to communicate with low-speed peripherals, such as sensors and real-time clocks. Specifically, enabling I2C does two things:

- Turns on the I2C interface on your Raspberry Pi.
- Loads the I2C kernel module at boot so that connected devices can communicate.

You can enable I2C from the desktop and using the interactive `raspi-config` TUI.

Desktop	raspi-config
---------	--------------

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **Interfaces** tab in the left-side menu.
3. Use the toggle next to **I2C** to turn it on or off.
4. Select **Close** in the bottom-right corner of the **Control Centre**.

Enable or disable the one-wire interface

Enabling the one-wire interface allows your Raspberry Pi to communicate with simple devices, such as temperature sensors. You can enable and disable the one-wire interface from the desktop and using the interactive `raspi-config` TUI.

[Desktop](#)[raspi-config](#)

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **Interfaces** tab in the left-side menu.
3. Use the toggle next to **1-Wire** to turn it on or off.
4. Select **Close** in the bottom-right corner of the **Control Centre**.
5. Reboot your Raspberry Pi for changes to take effect.

Enable or disable serial port

Enabling the Serial Port allows your Raspberry Pi to communicate over UART (Universal Asynchronous Receiver-Transmitter) with other devices, such as microcontrollers. There are two separate serial settings, which control different things:

- **Serial port hardware.** Enables the UART pins (TX/RX) for communication.
- **Serial console.** Sends kernel boot messages and provides a login shell over the serial connection.

You can enable and disable these serial port options separately from the desktop and using the interactive `raspi-config` TUI. For most hardware projects, you enable the serial port hardware and disable the serial console.

[Desktop](#)[raspi-config](#)

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **Interfaces** tab in the left-side menu.
3. Use the toggles to turn on the following settings as needed:
 - **Serial Port**, which enables and disables UART hardware.
 - **Serial Console**, which enables and disables login shell over serial.
4. Select **Close** in the bottom-right corner of the **Control Centre**.
5. Reboot your Raspberry Pi for changes to take effect.

Configure UARTs

UART (Universal Asynchronous Receiver-Transmitter) is a hardware communication protocol used for serial communication. UARTs on Raspberry Pi devices allow you to communicate with peripheral devices and to access the Linux terminal through a serial console.

Prerequisite information

To use UARTs on a Raspberry Pi, you must ensure that the correct UART is enabled, assigned to the appropriate pins, and not conflicting with other system services such as the serial console or Bluetooth.

UART types

Raspberry Pi supports two types of UART:

- **PL011.** A full-featured, 16550-compatible UART that provides hardware-timed serial communication.

- **Mini UART.** A lightweight UART that provides a simple and efficient serial interface.

Unlike **PL011**, the mini UART has no break detection, no framing errors detection, no parity bit, and no receive timeout interrupt. The mini UART also has smaller FIFOs. Combined with the lack of flow control, this makes the mini UART more prone to losing characters at higher baud rates. It's also generally less capable than PL011, mainly due to its baud rate link to the VPU clock speed. Further information about the mini UART can be found in the [SoC peripherals document](#).

Neither the mini UART nor the BCM2835 implementation of PL011 has DCD, DSR, DTR, or RI signals.

WARNING

All UARTs operate at 3.3 V; connecting to 5 V systems causes damage. For 5 V systems, use a level-shifter or a USB-to-3.3 V serial adaptor.

The available UART interfaces vary by Raspberry Pi model. The following table summarises the default UART configurations across common Raspberry Pi devices.

Table 2.
UART by Raspberry Pi model

Model	UARTs
Raspberry Pi Zero, 1, 2, and 3; Compute Module 1, 3, and 3+	UART0: PL011 UART1: mini UART
Raspberry Pi 4, 400, and Compute Module 4 and 4S	UART0: PL011 UART1: mini UART UART2 through to UART5: PL011 (disabled by default)
Raspberry Pi 5, 500, 500+, and Compute Module 5	UART0 through to UART4: PL011 (disabled by default) No mini UART

On Compute Modules, all UARTs are disabled by default and can be enabled through a Device Tree overlay, for example:

```
$ dtoverlay=uart1,txd1_pin=32,rx1_pin=33
```

Primary and secondary UARTs

The **primary UART** is the main serial interface that a Raspberry Pi exposes. This is the UART that you typically use when you connect external devices and for a Linux serial console. For all Raspberry Pis other than Raspberry Pi 5, the primary UART is routed to specific GPIO pins:

- **GPIO 14** (transmit) is pin 8 on the GPIO header.
- **GPIO 15** (receive) is pin 10 on the GPIO header.

On Raspberry Pi 5, the primary UART is exposed through the dedicated debug header instead.

The **secondary UART** is another serial interface available internally on the system; it isn't typically exposed through the GPIO header pins by default. On models with built-in wireless, this UART is commonly connected internally to the Bluetooth controller. This allows the system to communicate with Bluetooth hardware without using the external GPIO pins.

Table 3.
Primary and secondary UARTs

Model	Primary (Linux console)	Secondary (Bluetooth on Raspberry Pis with wireless)
Raspberry Pi Zero	UART0	UART1
Raspberry Pi Zero W and Zero 2 W	UART1	UART0
Raspberry Pi 1 and 2	UART0	UART1
Raspberry Pi 3 and 4	UART1	UART0

Model	Primary (Linux console)	Secondary (Bluetooth on Raspberry Pis with wireless)
Raspberry Pi 5	UART10	Dedicated UART
Compute Module 1, 3, 3+, and 4	UART0	UART1

Linux device names

In Linux, UARTs are accessed through files in `/dev/`. Each UART appears as a device node. These nodes act as interfaces that allow software to send and receive data by reading from or writing to the file.

Table 4.
Linux device nodes for UARTs

Device node	Type	Description
<code>/dev/ttyS0</code>	Hardware	Mini UART
<code>/dev/ttyAMA0</code>	Hardware	First PL011 (UART0)
<code>/dev/ttyAMA10</code>	Hardware	Debug UART on Raspberry Pi 5
<code>/dev/serial0</code>	Symbolic link	Primary UART: Depending on model, points to <code>/dev/ttyS0</code> , <code>/dev/ttyAMA0</code> , or <code>/dev/ttyAMA10</code> (Raspberry Pi 5).
<code>'/dev/serial1'</code>	Symbolic link	Secondary UART: Points to <code>/dev/ttyAMA0</code> or <code>/dev/ttyS0</code> .

Because different Raspberry Pi models use different hardware for their primary serial port, Raspberry Pi OS creates symbolic links that point to the correct hardware port.

For **Bookworm** and later, `/dev/serial1` might not exist by default. To enable it, add the following to `config.txt`:

```
$ dtparam=krnbt=off
```

This option might not work on all models in the future. Only use this option if there's no other alternative for your use case.

Mini UART and core frequency

Because the mini UART clock is derived from the VPU core clock, its reliability is affected when the core frequency changes dynamically; when the core clock frequency changes, the UART baud rate also changes. Variable core frequency disables the mini UART if it is primary or when Bluetooth is off. To enable it, set a fixed VPU clock:

```
$ enable_uart=1
```

The following table summarises the different configuration options and what happens in each case when you combine whether the mini UART is primary or secondary and whether the core clock is fixed or variable.

Table 5.
Mini UART configuration and core clock behaviour

Mini UART set to	Core clock	Result
Primary UART	Variable	Mini UART disabled
Primary UART	Fixed (<code>enable_uart=1</code>)	Mini UART enabled; core fixed at 250 MHz (or turbo frequency if forced)
Secondary UART	Variable	Mini UART disabled
Secondary UART	Fixed (<code>core_freq=250</code>)	Mini UART enabled

The system automatically determines whether to enable UART based on which hardware type is assigned as the primary UART. If the primary interface is PL011, the system defaults to 'on'. If the primary interface is the more sensitive mini UART, the system defaults to 'off' unless explicitly overridden to avoid issues with dynamic frequency scaling.

Table 6.
Default enable_uart behaviour by primary UART type

Primary UART	Default enable_uart
Mini UART	0
PL011 (UART0 or UART10)	1

Disable the Linux serial console

By default, the primary UART is assigned to the Linux console. If you want to use the primary UART for connecting to other serial devices, you must disable the console while leaving the UART enabled. You can do this from the desktop and using the interactive raspi-config TUI:



- 1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
- 2. Open the **Interfaces** tab in the left-side menu.
- 3. Turn on serial port hardware using the toggle next to **Serial Port**.
- 4. Turn off serial console using the toggle next to **Serial Console**.
- 5. Select **Close** in the bottom-right corner of the **Control Centre**.
- 6. Reboot your Raspberry Pi for changes to take effect.

Enable early console for Linux

For kernel debugging or monitoring early boot stages, the Linux console might not be immediately available. Enabling early console support for one of the UARTs can help to diagnose boot problems and hardware initialisation issues.

To enable early console support, add the relevant earlycon parameter to cmdline.txt, depending on your Raspberry Pi model and UART.

NOTE

earlycon=uart8250 outputs over the mini UART. earlycon=pl011 outputs over the full PL011 UART.

Table 7.
Early console configuration parameters by model

Model	Parameter string (add to cmdline.txt)
Raspberry Pi 5 (through the debug header)	earlycon=pl011,0x107d001000,115200n8
Raspberry Pi 4, 400, Compute Module 4, and 4S	earlycon=uart8250,mmio32,0xfe215040 earlycon=pl011,mmio32,0xfe201000
Raspberry Pi 2, 3, 3+, Zero 2 W, Compute Module 3, and 3+	earlycon=uart8250,mmio32,0x3f215040 earlycon=pl011,mmio32,0x3f201000
Raspberry Pi 1, Zero, Zero W, and Compute Module 1	earlycon=uart8250,mmio32,0x20215040 earlycon=pl011,mmio32,0x20201000

The baud rate defaults to 115200 bps.

WARNING

Selecting the wrong early console can prevent your Raspberry Pi from booting

Configure Device Tree Overlays for UARTs

You can use Device Tree Overlays to configure UART functionality, such as enabling additional UARTs, switching Bluetooth between PL011 and mini UART, or remapping GPIO pins used by UARTs.

When specifying an overlay in `config.txt`, use the overlay name only. The `-overlay.dts` suffix from the source filename is omitted.

You can find various UART Device Tree overlay definitions in the [kernel GitHub](#). The two most useful overlays are:

- `disable-bt`
- `miniuart-bt`

Disable Bluetooth and make UART0 primary

To disable the Bluetooth modem initialisation service and make the first PL011 (UART0) the primary UART:

1. Disable the Bluetooth modem service:

```
$ sudo systemctl disable hciuart
```

2. Open the configuration file:

```
$ sudo nano /boot/firmware/config.txt
```

3. Add the overlay to the configuration file:

```
dtoverlay=disable-bt
```

4. Save the file (**Ctrl+O**) and exit (**Ctrl+X**).
5. Reboot your Raspberry Pi for the changes to take effect.

Switch Bluetooth to use mini UART and make UART0 primary

To keep Bluetooth enabled but reassign it to the mini UART while making the first PL011 (UART0) the primary UART:

1. Open the configuration file:

```
$ sudo nano /boot/firmware/config.txt
```

2. Set a fixed VPU core clock in the configuration file by adding either:

- `force_turbo=1` (locks clock to maximum)
- `core_freq=250` (locks clock to standard speed)

3. Add the overlay to the configuration file:

```
dtoverlay=miniuart-bt
```

4. Save the file (**Ctrl+O**) and exit (**Ctrl+X**).
5. Reboot your Raspberry Pi for the changes to take effect

This can reduce the maximum usable baud rate for Bluetooth.

Enable additional UARTs on Raspberry Pi 4, 400, CM4, and CM4S

The overlays `uart2`, `uart3`, `uart4`, and `uart5` are used to enable the four additional UARTs on Raspberry Pi 4, 400, Compute Module 4, and Compute Module 4S.

For Raspberry Pi 5, 500, 500+ and Compute Module 5, `uart0-pi5`, `uart1-pi5`, `uart2-pi5`, `uart3-pi5` and `uart4-pi5` overlays are used.

To see the available pins and options for these UARTs, use the following help command, replacing `x` in `uartx` with the UART number you want to investigate:

```
dtoverlay -h uartx
```

Alternatively, refer to `/boot/firmware/overlays/README` for details on Device Tree Overlays.

Then, to apply an overlay, add it to `config.txt`:

1. Open the configuration file:

```
$ sudo nano /boot/firmware/config.txt
```

2. Add the overlay to `config.txt`, replacing `x` in `uartx` with the appropriate UART number:

```
dtoverlay=uartx
```

3. Save the file (**Ctrl+O**) and exit (**Ctrl+X**).
4. Reboot your Raspberry Pi for the changes to take effect

Performance

Edit this [on GitHub](#)

This section covers how to:

- **Configure CPU overclocking** on a Raspberry Pi 1 or 2. You can do this using the `raspi-config` TUI.
- **Set case fan behaviour** for a Raspberry Pi 4 with a case, which comes with an in-built fan. You can configure case fan behaviour from the desktop and using the `raspi-config` TUI.
- **Enable or disable USB current limit** for higher-power USB devices connected to your Raspberry Pi 5, 500, or 500+. You can enable or disable this setting from the desktop and using the `raspi-config` TUI.
- **Enable or disable overlay file system** to write-protect your storage device and make your Raspberry Pi read-only. You can do this from the desktop and using the `raspi-config` TUI.
- **Set PCIe port speed** for your Raspberry Pi 5, which is an advanced option that you can set using the `raspi-config` TUI and from the command line.
- **Configure shutdown behaviour**, which is applicable only to the Raspberry Pi 4 and 5 families of devices, is an advanced option used to save power when your Raspberry Pi is turned off. You can configure this using the `raspi-config` TUI.

Configure CPU overclocking

You can configure CPU overclocking on a Raspberry Pi 1 or 2 to boost performance for CPU-intensive tasks. Overclocking can increase heat output and power consumption.

To configure CPU overclocking on your Raspberry Pi 1 or 2 using the interactive `raspi-config` TUI:

1. Open the Terminal and run `sudo raspi-config`.
2. Use the keyboard to navigate to and select `4 Performance Options > P1 Overclock`.
3. Choose an overclock preset:
 - For Raspberry Pi 1, choose between:
 - **None**, which runs the CPU at 700 MHz, GPU at 250 MHz, and RAM at 400 MHz, with no extra voltage.
 - **Modest**, which runs the CPU at 800 MHz, GPU at 250 MHz, and RAM at 400 MHz, with no extra voltage.
 - **Medium**, which runs the CPU at 900 MHz, GPU at 250 MHz, and RAM at 450 MHz, and adds 2 overvolt.

- **High**, which runs the CPU at 950 MHz, GPU at 250 MHz, and RAM at 450 MHz, and adds 6 overvolt.
 - **Turbo**, which runs the CPU at 1000 MHz, GPU at 500 MHz, and RAM at 600 MHz, and adds 6 overvolt.
 - For Raspberry Pi 2, choose between:
 - **None**, which runs the CPU at 900 MHz, GPU at 250 MHz, and RAM at 450 MHz, with no extra voltage.
 - **High**, which runs the CPU at 1000 MHz, GPU at 500 MHz, and RAM at 500 MHz, and adds 2 overvolt.
4. Select <OK> and then <Finish> when you're done.
5. Reboot your Raspberry Pi for the changes to take effect.

Set case fan behaviour

The following instructions are only relevant to Raspberry Pi 4 for which you have a case, which comes with an attached fan.

Desktop

raspi-config

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **Performance** tab in the left-side menu.
3. Use the toggle next to **Case Fan** to turn it on.
4. Enter a number next to **FAN GPIO** to specify which GPIO the fan is connected to. For example, if you plugged your fan into GPIO 14, enter **14**.
5. Enter a number next to **Fan Temperature** to specify at what temperature in degrees Celsius you want the fan to turn on at. For example, if you want your fan to start working at 80°C, enter **80**.
6. Select **Close** in the bottom-right corner of the **Control Centre**.
7. Reboot your Raspberry Pi for the changes to take effect.

Enable or disable USB current limit

Applicable only if you're not using the official Raspberry Pi 5 power supply unit, the USB current limit controls how much power is supplied to USB ports on your Raspberry Pi. Removing the limit allows the use of higher-power USB devices. This setting is relevant to Raspberry Pi 5, 500, and 500+.

WARNING

Disabling the USB current limit can cause system instability, crashes, or data loss if connected USB devices require more power than the power supply can safely deliver.

Desktop

raspi-config

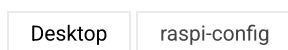
1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **Performance** tab in the left-side menu.
3. Use the toggle next to **Disable USB Current Limit** to turn it off and on.
4. Select **Close** in the bottom-right corner of the **Control Centre**.
5. Reboot your Raspberry Pi for the changes to take effect.

Enable or disable overlay file system

Overlay file system improves performance in write-heavy workloads by using RAM instead of slower storage. However, any changes will be lost when your Raspberry Pi reboots or powers off.

There are two separate overlay file system settings, which control different things:

- **Set the root file system to read-only.** This uses a temporary overlay stored in RAM. Any changes made to files outside of `/boot` while the system is running aren't permanently saved and disappear when you power off or reboot your Raspberry Pi.
- **Write-protect the boot partition.** This prevents any modifications to the `/boot` directory, which contains the firmware and configuration files needed to start your Raspberry Pi.



1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **Performance** tab in the left-side menu.
3. Select **Configure** next to **Overlay File System**.
4. Use the toggles to turn on the following settings as needed:
 - a. **Use Overlay**, which enables the read-only root file system overlay.
 - b. **Write-protect Boot Partition**, which prevents modifications to `/boot`.
5. Select **OK** in the bottom-right corner of the **Overlay File System** window
6. Select **Close** in the bottom-right corner of the **Control Centre**.
7. Reboot your Raspberry Pi for the changes to take effect.

Set PCIe port speed

Setting PCIe port speed is only relevant to Raspberry Pi 5. This is an advanced option in `raspi-config` that we don't recommend setting unless required by a PCIe-attached HAT. This is because Raspberry Pi 5 is certified for PCIe Gen 2.0 speeds and PCIe Gen 3.0 is thus disabled by default. Forcing a higher rate can lead to data corruption or system instability if the connected HAT or ribbon cable can't handle the increased frequency.

To enable PCIe Gen 3 using the interactive `raspi-config` TUI:

1. Open the Terminal and run `sudo raspi-config`.
2. Use the keyboard to navigate to and select `6 Advanced Options > A8 PCIe Speed`.
3. Select `Yes` or `No` in response to the question **Would you like PCIe Gen 3 to be enabled?**
4. Select `<OK>` to continue.
5. Reboot your Raspberry Pi for the change to take effect.

Configure shutdown behaviour

This is an advanced option in `raspi-config` used to save power when your Raspberry Pi is turned off. The default option depends on your Raspberry Pi model.

- On Raspberry Pi 400, 500, 500+, and Compute Module 5, the default is `Full power off`.
- On Raspberry Pi 4B, 5, and Compute Module 4, the default is `VPU sleep mode`.

To configure shutdown behaviour using the interactive `raspi-config` TUI:

1. Open the Terminal and run `sudo raspi-config`.
2. Use the keyboard to navigate to and select 6 Advanced Options > A11 Shutdown Behaviour.
3. Choose between:
 - B1 Full power off
 - B2 VPU sleep mode
4. Select <OK>` to continue and then <Finish> when you're done.
5. Reboot your Raspberry Pi for the change to take effect.

LED behaviour

[Edit this on GitHub](#)

On-board LEDs on Raspberry Pi devices provide a simple way to monitor system status and diagnose issues. Depending on the model, these LEDs indicate power state, SD card activity, and fault conditions, with some offering additional configurability for custom use.

Prerequisite information

Different Raspberry Pi models use different LED hardware configurations, and their exact behaviour and configurability differs accordingly.

The following table provides a high-level summary of power and activity LEDs, organised by Raspberry Pi model. Raspberry Pi 400 and 500 have two additional LEDs, described in more detail after this table.

Table 8.
Power and activity LEDs by Raspberry Pi model

Models	LED count	LED colours	MicroSD card activity indication
Raspberry Pi 1, 2, 3, and 4	2 (1 power LED and 1 activity LED)	Red and green	Green LED flashes during activity
Raspberry Pi Zero series	1	Green	LED flashes off during activity
Raspberry Pi 400	1	Green	No activity indication
Raspberry Pi 5, 500, 500+	1 bi-colour power and activity LED	Red or green depending on activity	Green LED flashes off during microSD activity

Raspberry Pi 1, 2, 3, and 4 LEDs

Raspberry Pi 1, 2, 3, and 4 use two distinct LEDs:

- Red for power (PWR), indicating normal operation with a stable 5 V power supply. The red LED is off or flickering when there is undervoltage.
- Green for microSD card activity (ACT), indicating no storage activity when off and microSD card access in progress when flashing.

When shutting down one of these Raspberry Pi models, the green activity LED briefly displays a series of short flashes to indicate that it's now safe to remove the USB PSU cable.

Raspberry Pi Zero LEDs

Raspberry Pi Zero, Zero W, and Zero 2 W use one green LED, which represents both power and microSD card activity. You can configure the LED to either:

- Flash off briefly when there is microSD card activity. This is the default behaviour.
- Remain lit when the board is powered on regardless of microSD card activity.

For instructions, see [Change LED behaviour on Raspberry Pi Zero](#).

Raspberry Pi 400 LEDs

Raspberry Pi 400 has a single green power LED alongside two keyboard-specific indicator lights in the following order:

- A num-lock LED, indicated on the keyboard with the number **1** inside a square.
- A caps-lock LED, indicated on the keyboard with the capital letter **A** inside a square.
- A power LED (described in [Power and activity LEDs by Raspberry Pi model](#)), indicated on the keyboard with a power symbol. This LED illuminates when power is applied and remains lit while the system is powered on. After the system has been shut down, all LEDs turn off.

For more detailed information about Raspberry Pi 400, see [Keyboard computers](#).

Raspberry Pi 500 LEDs

Raspberry Pi 500 uses a bi-colour (red and green) LED located above and to the left of the power key to indicate power and activity. It is located alongside two other indicator lights in the following order:

- A user-programmable LED, indicated on the keyboard with an icon consisting of a circle, square, and triangle. Its function isn't hardwired to system events, which means you can use the LED as a status light for your custom projects, for example, notification alerts and process indicators. For instructions, see [Configure Raspberry Pi 500 LEDs](#).
- A caps-lock LED, indicated on the keyboard with the capital letter **A** inside a square.
- A bi-colour power LED (described in [Power and activity LEDs by Raspberry Pi model](#)), indicated on the keyboard with a power symbol. This LED:
 - Illuminates red when power is first applied.
 - Changes to green as firmware startup progresses.
 - Remains green while the system is running.
 - Flashes off to indicate storage activity when booting from the microSD card.

After the operating system has been shut down, the LED changes to red to indicate the low-power standby state.

NOTE

The power key on Raspberry Pi 500 allows the computer to enter a soft-off state. In this state, the CPU and most system activity stops, but the power rails remain active and the system can be restarted by pressing the power key again, or by removing and restoring power as normal.

For more detailed information about Raspberry Pi 500, see [Keyboard computers](#).

Raspberry Pi 5 and 500+ LEDs

Raspberry Pi 5 and 500+ use a bi-colour (red and green) LED to indicate power and activity.

On Raspberry Pi 5, the power LED is located next to the power button; if assembled into a Raspberry Pi 5 case, the power button and LED appear integrated behind a single pressable lens.

On Raspberry Pi 500+, the power LED is located underneath the power key.

On both Raspberry Pi 5 and 500+, the bi-colour power LED (described in [Power and activity LEDs by Raspberry Pi model](#)):

- Illuminates red when power is first applied.
- Changes to green as firmware startup progresses.
- Remains green while the system is running.

- Flashes off to indicate microSD card activity. It doesn't flash if you're booting from the internal SSD on a Raspberry Pi 500+, or from an SSD on an M.2 HAT+ on a Raspberry Pi 5.

After the operating system has been shut down, the LED changes to red to indicate the low-power standby state.

NOTE

The power button on Raspberry Pi 5 and the power key on Raspberry Pi 500+ allows the computer to enter a soft-off state. In this state, the CPU and most system activity stops, but the power rails remain active and the system can be restarted by pressing the power button or key again, or by removing and restoring power as normal.

For more detailed information about Raspberry Pi 500+, see [Keyboard computers](#).

Change LED behaviour on Raspberry Pi Zero

Raspberry Pi Zero, Zero W, and Zero 2 W have a single power LED for indicating power and storage activity.

You can use the interactive `raspi-config` TUI to change the behaviour of the power LED on your Raspberry Pi Zero, Zero W, or Zero 2 W:

1. Open the Terminal and run `sudo raspi-config`.
2. Use the keyboard to navigate to and select `1 System Options > S8 Power LED`.
3. Select `Yes` or `No` in response to the question **Would you like the power LED to flash during disk activity?**
 - Selecting `Yes` configures the LED to flash when there's storage activity.
 - Selecting `No` keeps the LED lit when the board is powered on.
4. Select `<OK>` to confirm this choice and then `<Finish>` when you're done.

Configure Raspberry Pi 500 LEDs

The user-configurable LED on Raspberry Pi 500 can be controlled through the `pinctl` tool, which is used to interact with GPIO pins and peripherals.

- `pinctl USER_LED op dh` turns the user LED on.
- `pinctl USER_LED op dl` turns the user LED off.

You can also use Python to call `pinctl`:

```
import subprocess
from time import sleep

while True:
    subprocess.run(['pinctl', 'USER_LED', 'op', 'dh'])
    print("On")
    sleep(1)
    subprocess.run(['pinctl', 'USER_LED', 'op', 'dl'])
    print("Off")
    sleep(1)
```

Check LED warning codes

If a Raspberry Pi device fails to boot or shuts down unexpectedly, the on-board LED often provides a diagnostic signal in the form of a specific flashing pattern. Long flashes always occur before short ones that indicate the exact issue; sometimes there are no long flashes at all. In most cases, the pattern repeats after a two-second pause.

Long flashes	Short flashes	Status
0	3	Generic failure to boot
0	4	<code>start*.elf</code> not found
0	7	Kernel image not found
0	8	SDRAM failure

Long flashes	Short flashes	Status
0	9	Insufficient SDRAM
0	10	In HALT state
1	2	SD card overcurrent detected
2	1	Partition not FAT
2	2	Failed to read from partition
2	3	Extended partition not FAT
2	4	File signature/hash mismatch (Raspberry Pi 4 and 5)
3	1	SPI EEPROM error (Raspberry Pi 4 and 5)
3	2	SPI EEPROM is write protected (Raspberry Pi 4 and 5)
3	3	I2C error (Raspberry Pi 4 and 5)
3	4	Secure-boot configuration isn't valid
4	3	RP1 not found
4	4	Unsupported board type
4	5	Fatal firmware error
4	6	Power failure type A
4	7	Power failure type B

Boot behaviour

[Edit this on GitHub](#)

Boot behaviour determines what your Raspberry Pi loads when it starts up. You can configure it to either launch a full desktop graphical user interface (GUI) or start in a text-based command-line interface (CLI), also referred to as console mode. For instructions, see [Boot to console or desktop](#).

You can also control what is displayed during boot. Specifically, you can turn the standard Plymouth splash screen on or off, and you can replace the default early boot Raspberry Pi splash screen with a custom early boot splash screen. For instructions, see [Configure splash screens](#).

This section also covers the kernel command line, which is the primary mechanism for passing low-level instructions to the Linux kernel during the boot process. While the bootloader initialises the hardware (through `config.txt`), the kernel command line defines how the OS itself behaves when it takes over (through `cmdline.txt`). For instructions, see [Configure the kernel command line](#).

NOTE

If you're looking for information about auto login, see [Require a password from startup](#).

Boot to console or desktop

You can choose whether to boot your Raspberry Pi to console or to desktop when you start up your Raspberry Pi from the desktop and using the interactive `raspi-config` TUI.

Desktop

raspi-config

1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **System** tab in the left-side menu.
3. Next to **Boot**, choose either **To desktop** or **To CLI**. Selecting **To CLI** turns off desktop auto login, requiring you to enter your password if you change this setting back to **To desktop** in the future.
4. Select **Close** in the bottom-right corner of the **Control Centre**.

5. Reboot your Raspberry Pi to apply your changes.

Configure splash screens

Raspberry Pi OS can display two types of splash screen during boot:

- **A standard Plymouth splash screen.** This is a graphical boot that you can turn on or off. If the standard Plymouth splash screen is turned off, the system shows text-based console log messages instead.
- **A custom early boot splash screen.** A fullscreen image that appears before the standard Plymouth splash screen, replacing the Raspberry Pi logos.

Show or remove the Plymouth splash screen

You can choose whether to show the Raspberry Pi graphical splash screen instead of console log messages (text) on boot using either the desktop GUI or the interactive `raspi-config` TUI.



1. Select the Raspberry Pi icon in the top-left corner of the system tray and then go to **Preferences > Control Centre**.
2. Open the **System** tab in the left-side menu.
3. Use the toggle next to **Splash Screen** to add or remove the graphical splash screen at boot.
4. Select **Close** in the bottom-right corner of the **Control Centre**.

Customise the early boot splash screen

The early boot splash screen is a fullscreen image that appears in the initial seconds of the boot process, appearing before the standard Plymouth splash screen. It provides visual feedback that the system is loading; you can choose whether to show this screen and you can customise it.

The fullscreen splash image must meet the following file requirements:

- **Maximum dimensions.** 1920 × 1080 pixels.
 - If the image is smaller than the screen, the remaining area around your image is filled with a solid color taken from the top-left pixel of your image (0, 0).
 - Scaling isn't applied. If the screen is smaller than the image, the image is clipped and centred.
- **Maximum colours.** 224 unique colours.
- **Colour depth.** 24-bit (8 bits per channel, no alpha channel).
- **Format.** Uncompressed TGA.

You can use ImageMagick's `convert` command to convert an image to the TGA format expected by the kernel. For example, to convert `image.png` to a suitably-formatted `splash-image.tga` file:

```
$ sudo apt install imagemagick
$ convert image.png -colors 224 -depth 8 -type TrueColor -alpha off -compress none -define tga:bits-per-sample=8 splash-image.tga
```

There are two ways to enable a fullscreen splash image on your Raspberry Pi. Depending on your technical expertise, you can either:

- **Use the APT package.** This method is more straightforward, allowing you to enable the splash screen with minimal configuration. You install a pre-made package (`raspi-splash-screen-support`) that automates most of the setup. It copies your image, sets kernel parameters, and configures the system for you.

- **Edit the `cmdline.txt` system file.** This method is more technical, but gives you control over every step. You manually configure the splash image on the command line and then update your `initramfs` to contain the splash image.

Use the APT package

Install the `rpi-splash-screen-support` package to simplify the set up of fullscreen splash images. The package provides the `configure-splash` tool, which sets the necessary kernel parameters and updates your `initramfs`.

1. Install the package using the following command:

```
$ sudo apt install rpi-splash-screen-support
```

2. After the APT package is installed, run the following command to configure your splash image, replacing `<splash-image.tga>` with the name or path of the image file. This path can be relative or absolute:

```
$ sudo configure-splash <splash-image.tga>
```

3. Run the following command to reboot your Raspberry Pi and see the splash image:

```
$ sudo reboot
```

If the image appears upside down when booting, edit the `convert` command to add the `-flip` flag before the `splash-image.tga`. Then run `configure-splash` to update the image with the correct orientation.

Edit the `cmdline.txt` system file

The following method requires editing system files and manually updating the `initramfs`.

1. Use the following command to open `/boot/firmware/cmdline.txt` in a text editor as an administrator:

```
$ sudo nano /boot/firmware/cmdline.txt
```

2. Disable on-screen console messages to prevent boot messages from covering your splash image. Edit `cmdline.txt` to remove the following:

- `console=tty1`.
- `quiet` (if present).

3. Edit `cmdline.txt` to add the following parameters:

- `fullscreen_logo_name=logo.tga fullscreen_logo=1` to enable fullscreen splash.
- `vt.global_cursor_default=0` to remove the flashing cursor in the splash image.

Your entry should end with something like the following:

```
fullscreen_logo_name=logo.tga fullscreen_logo=1 vt.global_cursor_default=0
```

4. Place the image file in the correct location.

- The kernel expects the image file in `/lib/firmware`.
- The TGA file is read from the `initramfs` during boot.
- The filename must match `fullscreen_logo_name` in `cmdline.txt`.

For example code for embedding TGA images in `initramfs`, see the [Raspberry Pi splash screen support tool](#) in GitHub.

NOTE

Step 2 doesn't prevent `getty` from launching a login prompt. Your splash image appears during boot, but when the system is ready for login, the console takes over, and the splash disappears. `getty` clears the splash screen and writes the login prompt over it when the system is ready.

Configure the kernel command line

When a Raspberry Pi (or any Linux system) boots, the kernel is given a set of startup parameters called the **kernel command line**. The kernel command line allows you to configure various aspects of the system, from troubleshooting USB lag to customising your display resolution.

Access and edit `cmdline.txt`

The Linux kernel receives a set of parameters during boot. On a Raspberry Pi, these are stored in a specific file in the boot partition called `cmdline.txt`. Use the following command to modify the boot parameters:

```
$ sudo nano /boot/firmware/cmdline.txt
```

IMPORTANT

All parameters in `cmdline.txt` must stay on the same single line. Don't use newlines or press **Enter** or **Return** on your keyboard between commands because the kernel ignores anything after the first line.

View current settings

Use the following command to view the Linux kernel's current boot parameters:

```
$ cat /proc/cmdline
```

This reads a specific file in `/proc` that reflects the live kernel state. This might not exactly match your original `cmdline.txt`. This is because Raspberry Pi firmware makes changes to the kernel command line before launching the kernel.

Define core parameters

The following standard entries define how your Raspberry Pi identifies its hardware and loads the operating system.

Standard entry	Description	Notes
console	Defines where boot messages are sent.	There are usually two entries: <code>console=serial0,115200</code> (serial mode) <code>console=tty1</code> (main screen)
root	Defines the location of the root filesystem (where the OS lives).	For example, <code>root=/dev/mmcblk0p2</code> means multimedia card block-device 0, partition 2 (for the second partition on the microSD card).
rootfstype	Specifies the format of the partition (the type of filesystem the <code>rootfs</code> uses).	For example, <code>rootfstype=ext4</code> .
quiet	Sets the default kernel log level to <code>KERN_WARNING</code> .	This suppresses all but very serious log messages during boot.

Additional kernel command line entries

This section contains some of the other entries you can use in the kernel command line. This list is not exhaustive.

Entry	Description
splash	Tells the boot to use a splash screen through the Plymouth module.
plymouth.ignore-serial-consoles	Normally, Plymouth suppresses boot messages on serial consoles. This entry forces Plymouth to ignore serial consoles so boot messages remain visible.
dwc_otg.lpm_enable=0	Disables Link Power Management (LPM) in the <code>dwc_otg</code> USB driver used by the processor's USB controller. On

Entry	Description
	Raspberry Pi 4 and later, this controller only affects the USB-C power input and not the USB-A ports.
dwc_otg.speed=1	Sets the USB controller speed to full speed (USB 1.0). Useful for troubleshooting USB issues but slower than high speed (USB 2.0). Only use this entry when diagnosing problems.
smc95xx.turbo_mode	Applicable only to Raspberry Pi 1B, 1B+, 2B, 3B, and 3B+. Enables or disables turbo mode for the wired Ethernet driver. Setting smc95xx.turbo_mode=N disables turbo mode.
usbhid.mousepoll	Adjusts mouse polling interval. Setting usbhid.mousepoll=0 might help with slow or erratic wireless mice.
drm.edid_firmware=HDMI-A-1:edid/your_edid.bin	Overrides the monitor's built-in EDID using a custom EDID file located in /usr/lib/firmware/edid/.

Firmware

Edit this on GitHub

This section covers how to:

- **Change the boot order** on newer Raspberry Pi models using the interactive raspi-config TUI.
- **Update the bootloader** using the interactive raspi-config TUI and directly from the command line.
- **Manage bootloader configuration** directly from the command line.
- **Enable or disable the network install UI** using the interactive raspi-config TUI.

Change the boot order

Boot order determines where your Raspberry Pi tries to boot from. On Raspberry Pi 4 and later, you can specify whether to boot from USB or network when no SD card or SSD has been detected. The default is to boot from the SD card first.

To change the boot order using the raspi-config TUI:

1. Open the Terminal and run `sudo raspi-config`.
2. Navigate to and select `6 Advanced Options > A4 Boot Order`.
3. Choose from one of the following options:
 - `B1 SD Card Boot` (recommended) boots the system from the SD card first. If no bootable SD card is found, the bootloader then tries NVMe and then USB.
 - `B2 NVMe/USB Boot` boots from NVMe storage first. If no bootable NVMe device is found, the bootloader then tries USB and then the SD card.
 - `B3 Network Boot` boots from the SD card first. If no bootable SD card is found, the bootloader then tries to boot over the network (PXE).
4. The Terminal briefly runs some code before returning to the TUI confirming that your selection has been implemented. Select `<OK>` and then `<Finish>`.

Update the bootloader

On Raspberry Pi OS, the bootloader version is automatically updated when important bug fixes and improvements are released. CM4 and CM4S don't support automatic bootloader updates because their boot ROM can't load the `recovery.bin` file from eMMC. For these models, manual update tools such as `rpiboot` and `flashrom` are recommended. For details, run:

```
$ `rpi-eeeprom-update -h`
```

On Raspberry Pi 4 and later, you can manually switch to the latest boot ROM software. Alternatively, you can revert to the factory default. Specifically, bootloader version updates are supported on:

- Flagship models since Raspberry Pi 4B.
- Compute Modules since CM5.
- All keyboard computers (Raspberry Pi 400, 500, and 500+).

You can update the bootloader by switching between the latest and the default bootloader using `raspi-config` TUI. Alternatively, use the command line to update the bootloader without switching.

If, instead of updating the bootloader version, you want to reset the bootloader, see [Reset the bootloader](#).

`raspi-config` CLI

First, open the Terminal and run the following code to update Raspberry Pi OS, getting the latest version of the `rpi-eeeprom` package:

```
$ sudo apt update && sudo apt full-upgrade
```

Then:

1. Use the keyboard to navigate to and select `6 Advanced Options > A5 Bootloader Version`.
2. Select one of the following release tracks:
 - `E1 Latest` for the latest bootloader release and new bootloader features. This option is more appropriate for advanced users who want the latest, untested functionality.
 - `E2 Default` for stable, factory-default settings. This option provides critical bug fixes, hardware support, and the latest tested features.
3. Select `Yes` to confirm your choice.
4. Reboot your Raspberry Pi to apply your changes.

Manage bootloader configuration

This section provides instructions for reading the bootloader version and managing the bootloader configuration from the command line. Developers and system administrators can find the scripts and pre-compiled binaries used to create `rpi-eeeprom` images in the [rpi-eeeprom GitHub repository](#).

Check bootloader version and configuration

Run the following command to check the current bootloader version:

```
$ sudo rpi-eeeprom-update
```

The output tells you what version is currently installed on your Raspberry Pi (CURRENT), what the most recent available release is (LATEST), and the release track used by the system (RELEASE). For example:

```
*** UPDATE AVAILABLE ***
BOOTLOADER: update available
CURRENT: Thu 18 Jan 13:59:23 UTC 2024 (1705586363)
LATEST: Mon 22 Jan 10:41:21 UTC 2024 (1705920081)
RELEASE: latest (/lib/firmware/raspberrypi/bootloader-2711/latest)
        Use raspi-config to change the release.

VL805_FW: Using bootloader EEPROM
VL805: up to date
CURRENT: 000138c0
LATEST: 000138c0
```

Read the current bootloader configuration

Run the following command to view the configuration used by the current running bootloader:

```
$ rpi-eeprom-config
```

Run the following command to read the configuration from a bootloader image:

```
$ rpi-eeprom-config pieeprom.bin
```

Edit the bootloader configuration

The following commands load the current bootloader configuration into a text editor. The text editor is selected by the `EDITOR` environment variable.

When the editor is closed, `rpi-eeprom-config` applies the updated configuration to the latest available bootloader release and uses `rpi-eeprom-update` to schedule an update when the system is rebooted:

```
$ sudo -E rpi-eeprom-config --edit
$ sudo reboot
```

If the updated configuration is identical or empty, the bootloader remains unchanged.

Apply a saved configuration

Run the following commands to apply a saved configuration file (for example, `boot.conf`) to the latest bootloader release.

```
$ sudo rpi-eeprom-config --apply boot.conf
$ sudo reboot
```

This schedules the changes to take effect after reboot.

Enable or disable the network install UI

The user interface (UI) for network install controls whether your Raspberry Pi displays a menu for network-based OS installation when no local boot media is detected. On Raspberry Pi 4 and later, enabling this option allows you to interact with the bootloader to perform a network installation (PXE boot). Disabling this option keeps the network install process hidden.

To change the network install UI setting using the interactive `raspi-config` TUI:

1. Open the Terminal and run `sudo raspi-config`.
2. Use the keyboard to navigate to and select `6 Advanced Options > A9 Network Install UI`.
3. Choose between one of the following options:
 - `B1 Always`, which always displays the network install UI for a few seconds after power on.
 - `B2 On demand`, which displays the network install UI if the **Shift** key on your keyboard is pressed or if there is an error.
4. Select `<OK>` and then `<Finish>` when you're done.

External storage

Edit this [on GitHub](#)

You can connect your external hard disk, SSD, or USB stick to any of the USB ports on the Raspberry Pi, and mount the file system to access the data stored on it.

By default, your Raspberry Pi automatically mounts some of the popular file systems such as FAT, NTFS, and HFS+ at the `/media/pi/<HARD-DRIVE-LABEL>` location.

NOTE

Raspberry Pi OS Lite does not implement automounting.

To set up your storage device so that it always mounts to a specific location of your choice, you must mount it manually.

Mount a storage device

You can mount your storage device at a specific folder location. It is conventional to do this within the `/mnt` folder, for example `/mnt/mydisk`. Note that the folder must be empty.

Plug the storage device into a USB port on the Raspberry Pi, and list all the disk partitions on the Raspberry Pi using the following command:

```
$ sudo lsblk -o UUID,NAME,FSTYPE,SIZE,MOUNTPOINT,LABEL,MODEL
```

The Raspberry Pi uses mount points `/` and `/boot/firmware/`. Your storage device will show up in this list, along with any other connected storage.

Use the `SIZE`, `LABEL`, and `MODEL` columns to identify the name of the disk partition that points to your storage device. For example, `sda1`. The `FSTYPE` column contains the filesystem type. If your storage device uses an exFAT file system, install the exFAT driver:

```
$ sudo apt update
$ sudo apt install exfat-fuse
```

If your storage device uses an NTFS file system, you will have read-only access to it. If you want to write to the device, you can install the `ntfs-3g` driver:

```
$ sudo apt update
$ sudo apt install ntfs-3g
```

Run the following command to get the location of the disk partition:

```
$ sudo blkid
```

For example, `/dev/sda1`.

Create a target folder to be the mount point of the storage device. The mount point name used in this case is `mydisk`. You can specify a name of your choice:

```
$ sudo mkdir /mnt/mydisk
```

Mount the storage device at the mount point you created:

```
$ sudo mount /dev/sda1 /mnt/mydisk
```

Verify that the storage device is mounted successfully by listing the contents:

```
$ ls /mnt/mydisk
```

Automatically mount a storage device

You can modify the `fstab` file to define the location where the storage device will be automatically mounted when the Raspberry Pi starts up. In the `fstab` file, the disk partition is identified by the universally unique identifier (UUID).

Get the UUID of the disk partition:

```
$ sudo blkid
```

Find the disk partition from the list and note the UUID (for example, `5C24-1453`). Open the `fstab` file using a command line editor such as `nano`:

```
$ sudo nano /etc/fstab
```

Add the following line in the `fstab` file:

```
UUID=5C24-1453 /mnt/mydisk fstype defaults,auto,users,rw,nofail 0 0
```

Replace `fstype` with the type of your file system, which you found when you went through the steps above, for example: `ntfs`.

If the filesystem type is FAT or NTFS, add `,umask=000` immediately after `nofail` – this will allow all users full read/write access to every file on the storage device.

Now that you have set an entry in `fstab`, you can start up your Raspberry Pi with or without the storage device attached. Before you unplug the device you must either shut down the Raspberry Pi, or manually unmount it.

NOTE

If you do not have the storage device attached when the Raspberry Pi starts, it will take an extra 90 seconds to start up. You can shorten this by adding `,x-systemd.device-timeout=30` immediately after `nofail`. This will change the timeout to 30 seconds, meaning the system will only wait 30 seconds before giving up trying to mount the disk.

For more information on each Linux command, refer to the specific manual page using the `man` command. For example, `man fstab`.

Unmount a storage device

When the Raspberry Pi shuts down, the system takes care of unmounting the storage device so that it is safe to unplug it. If you want to manually unmount a device, you can use the following command:

```
$ sudo umount /mnt/mydisk
```

If you receive an error that the 'target is busy', this means that the storage device was not unmounted. If no error was displayed, you can now safely unplug the device.

Dealing with 'target is busy'

The 'target is busy' message means there are files on the storage device that are in use by a program. To close the files, use the following procedure.

Close any program which has open files on the storage device. If you have a terminal open, make sure that you are not in the folder where the storage device is mounted, or in a sub-folder of it.

If you are still unable to unmount the storage device, you can use the `lsof` tool to check which program has files open on the device. You need to first install `lsof` using `apt`:

```
$ sudo apt update
$ sudo apt install lsof
```

To use `lsof`:

```
$ lsof /mnt/mydisk
```

Operational security

[Edit this on GitHub](#)

This section summarises common ways to improve the security of your Raspberry Pi. If you're looking for password instructions, see [User access and management](#).

Update Raspberry Pi OS

To get the latest security features, we recommend that you keep your software up to date. For more information, see [Update your current Raspberry Pi software](#) in the [Raspberry Pi OS](#) page.

Only the most recent major release of Raspberry Pi OS contains all the latest security fixes. We therefore recommend that you also upgrade to the most recent major release when it becomes available (for example, from Bookworm to Trixie). This is different from *updating* your Raspberry Pi OS, which installs the latest packages and security fixes within your current release without changing to a new major version. For instructions on upgrading your OS, see [Upgrade to a new major version](#).

Automatically update your SSH server

If you use SSH to connect to your Raspberry Pi, it can be worthwhile to add a cron job that specifically updates the SSH server. The following command, perhaps run as a daily cron job, ensures you have the latest SSH security fixes promptly, independent of your normal update process.

```
$ apt install openssh-server
```

Improve SSH security

SSH is a common way to remotely access a Raspberry Pi. By default, SSH requires a username and password. To make SSH even more secure, use **key-based authentication**.

Enable and disable SSH users

You can also **allow** or **deny** specific users by altering the `sshd` configuration.

```
$ sudo nano /etc/ssh/sshd_config
```

Add, edit, or append to the end of the file the following line, which contains the usernames you wish to allow to log in:

```
AllowUsers alice bob
```

You can also use `DenyUsers` to specifically stop some usernames from logging in:

```
DenyUsers jane john
```

After the change, restart the `sshd` service with the following command to put your changes into effect:

```
$ sudo systemctl restart ssh
```

Use a firewall

A firewall is a network security system that monitors and controls incoming and outgoing network traffic based on a defined set of security rules. This section introduces **Uncomplicated Firewall (UFW)**, a firewall tool that offers a simplified command-line interface for controlling network traffic through `iptables`. There are also some graphical user interface (GUI) frontends available for UFW, such as GFW, but `ufw` commands are typically run on the command line. This section provides an overview of, and basic instructions for, the UFW command-line interface.

Uncomplicated Firewall (UFW)

Network traffic is organised by port numbers, which are numerical labels used to direct data to specific applications. Firewalls control traffic by opening and closing particular ports to allow or block certain types of communication.

On Linux systems, network traffic is filtered through the kernel's firewall subsystem, called **netfilter**. Netfilter implements packet filtering; it blocks or allows data packets to pass through your system based on things like their source IP address, destination IP address, port number, protocol, and direction.

System administrators can interact with netfilter through `iptables`, which is the main command-line tool for defining packet filtering rules. However, `iptables` can be complex. **UFW (Uncomplicated Firewall)**, the default firewall tool used by Ubuntu, simplifies firewall configuration and management. It acts as a wrapper for `iptables`, allowing system administrators to set firewall rules without dealing with the complexity of raw `iptables` commands.

UFW commands

UFW is a command-line program used to define firewall access rules. The typical syntax for `ufw` commands is: `sudo ufw [--dry-run] <command> [rule]`.

- All `ufw` commands require superuser privileges, and so must be preceded with `sudo`.

- Optionally, you can add `--dry-run` to simulate the `ufw` command without making actual changes.
- The `<command>` is the main action that you want to perform, such as `allow`.
- Depending on the command, you can optionally specify a `rule` to apply to the command, such as `allow 22`.

For example, `sudo ufw --dry-run allow 22` shows you the would-be outcome of allowing traffic on port 22, including the entire set of rules if the change were made.

Install and enable UFW

To use UFW, you must first install it onto your Raspberry Pi device. After installation, UFW is present but disabled, meaning that it doesn't block any traffic yet. This allows you to configure default access rules without locking yourself out. This is especially important if you're connected to your Raspberry Pi device over SSH (or any other remote method); to avoid being locked out, you must allow remote access before enabling UFW.

If you're connected over SSH, the recommended flow of actions is as follows:

Step	Description	Command
1	Update your package list.	<code>sudo apt-get update</code>
2	Install UFW. Before UFW is enabled, the output says that UFW is inactive.	<code>sudo apt install ufw</code>
3	Verify the installation by checking the status. For information about this command, see Rule management .	<code>sudo ufw status</code>
4	Optionally (but recommended), configure default policies, such as denying all incoming traffic. Defaults apply globally.	<code>sudo ufw default deny incoming</code>
5	To maintain remote access, allow SSH (before enabling UFW), either by specifying the service name (<code>ssh</code>) or the default port (<code>22/tcp</code>).	<code>sudo ufw allow ssh</code> OR <code>sudo ufw allow 22/tcp</code>
6	Optionally, if you're running a web server, allow HTTP and HTTPS traffic, either by specifying the service names (<code>http</code> and <code>https</code>) or their default ports (<code>80/tcp</code> and <code>443/tcp</code> , respectively).	<code>sudo ufw allow http</code> <code>sudo ufw allow https</code> OR <code>sudo ufw allow 80/tcp</code> <code>sudo ufw allow 443/tcp</code>
7	Enable UFW to activate the firewall and configure it to start up on boot.	<code>sudo ufw enable</code>
8	Verify the status of the firewall and review the configured rules. For information about this command, see Rule management .	<code>sudo ufw status verbose</code>

You can disable UFW at any time, which stops it from starting up on boot. Use the following command to disable the UFW firewall:

```
$ sudo ufw disable
```

Use allow and deny rules

This section lists the basic commands for allowing and denying traffic through specified service names, ports, and services. The general `ufw` syntax for `allow` and `deny` commands are as follows:

- Allow access to a particular port or service: `sudo ufw allow [rule]`
- Deny access to a particular port or service: `sudo ufw deny [rule]`

The following table provides specific examples.

Command	Description
<code>sudo ufw allow 22</code>	Allows access to port 22 through the firewall. This opens the port for all protocols (TCP and UDP).
<code>sudo ufw allow 22/tcp</code>	Allows TCP only on port 22 .
<code>sudo ufw allow ssh</code>	Allows the SSH service without specifying the port (SSH is usually port 22/tcp).
<code>sudo ufw deny 22</code>	Denies access to port 22 through the firewall. This closes the port for all protocols (TCP and UDP).
<code>sudo ufw deny 22/tcp</code>	Denies TCP only on port 22 .
<code>sudo ufw deny ssh</code>	Denies the SSH service without specifying the port (SSH is usually port 22/tcp).

Advanced UFW rules

UFW can do more than basic allow and deny rules. It can also:

- **Allow or block a specific IP address** from a specific port.
- **Specify traffic direction** (incoming and outgoing).
- **Rate limit connection attempts** to help protect against brute-force or DDoS attacks.
- **Specify network interfaces** to apply rules specifically to wired or wireless networks.

For more information about these options, run the following command: `man ufw`. This displays the UFW manual page in your terminal, which includes documentation on UFW syntax, options, rule formatting, and examples.

Allow or block a specific IP address

You can allow or deny access from a specific IP address to a specific port. This is useful if you want to block suspicious hosts or allow trusted hosts only. For example, the following command denies access to port 30 from IP address 192.168.2.1

```
$ sudo ufw deny from 192.168.2.1 to any port 30
```

Specify traffic direction

To control how your system sends or receives network traffic, perhaps to block threats to your system (incoming) or protect system resources (outgoing), you can prevent traffic from coming in one or both directions. Specifically, you can add `out` (for outgoing traffic) or `in` (for incoming traffic) to one of the following commands:

- `allow`, which allows the traffic to flow in the specified direction.
- `deny`, which silently drops the traffic flowing in the specified direction.
- `reject`, which actively refuses the traffic flowing in the specified direction.

For example, the following command prevents all outgoing traffic on the SMTP port (usually port 25/TCP):

```
$ sudo ufw reject out smtp
```

Rate limit connection attempts

To protect against brute-force and DDoS attacks, the `limit` command temporarily blocks any IP address that attempts to make too many connections in a short period of time (approximately 6 or more times within 30 seconds). For example, the following command watches how often an IP address attempts to open a new connection to SSH (usually port 22/tcp) and applies the rate limit.

```
$ sudo ufw limit ssh/tcp
```

Specify network interfaces

You can set firewall rules to apply only to a specific network connection (`eth0` for wired networks or `wlan0` for Wi-Fi). For example, the following command allows

incoming TCP traffic on port 80 only on the eth0 network interface:

```
$ sudo ufw allow in on eth0 to any port 80 proto tcp
```

Rule management

UFW processes rules in order, and so the order of rules matters. The following table lists useful commands to help you inspect your current firewall configuration and remove rules when necessary.

Command	Description
sudo ufw status	Displays whether UFW is active and lists all current settings for the firewall (which ports and services are allowed and denied).
sudo ufw status verbose	Displays additional details, such as default policies and logging settings.
sudo ufw show added	Displays only user-added rules (excluding internal and system rules). This is useful if you want to review changes or compare configurations across systems.
sudo ufw status numbered	Displays rules with their assigned index number. This is useful if you want to delete a specific rule.
sudo ufw delete <number>	Deletes the rule that you specify based on its assigned index number. UFW then rennumbers the remaining entries. You can rerun sudo ufw status numbered to confirm the rule was removed.

Block suspicious activity with fail2ban

When using a Raspberry Pi as a server, you must create deliberate holes in your firewall to allow server traffic. Fail2ban can help secure your server. Fail2ban examines log files and checks for suspicious activity, like multiple brute-force login attempts. It saves you having to manually check log files for intrusion attempts and then update the firewall (via iptables) to prevent them.

To install fail2ban, run the following command:

```
$ sudo apt install fail2ban
```

On installation, Fail2ban creates /etc/fail2ban/jail.conf. To enable Fail2ban, copy jail.conf to jail.local:

```
$ sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
```

Inside this configuration file are a set of default options, together with options for checking specific services for abnormalities. To examine the rules used for ssh, open jail.local in an editor:

```
$ sudo nano /etc/fail2ban/jail.local
```

Create the [ssh] section if it does not already exist and add the following lines to the section:

```
[ssh]
enabled = true
port = ssh
filter = sshd
backend = systemd
maxretry = 6
```

This enables Fail2ban checks for suspicious ssh activity, including system log checks, and allows six retries before blocking activity.

The [default] section in this same file defines the default banning action, iptables-multiport, which runs the /etc/fail2ban/action.d/iptables-multiport.conf file when the detection threshold is reached:

```
# Default banning action (for example, iptables, iptables-new,
# iptables-multiport, shorewall, etc) It is used to define
# action_* variables. Can be overridden globally or per
```

```
# section within jail.local file
banaction = iptables-multiport
```

Multiport bans all access on all ports. The `action.d` folder contains a number of alternative action configuration files you can use to customise your server's response to suspicious activity.

For instance, to permanently ban an IP address after three failed attempts, change the `maxretry` value in the `[ssh]` section to 3 and set the `bantime` to a negative number:

```
[ssh]
enabled = true
port    = ssh
filter  = sshd
backend = systemd
maxretry = 3
bantime = -1
```

Further reference

[Edit this on GitHub](#)

This section provides background information on aspects of the Raspberry Pi boot process and hardware configuration.

boot folder contents

Raspberry Pi OS stores boot files on the first partition of the SD card, formatted with the FAT file system.

On startup, each Raspberry Pi loads various files from the boot partition in order to start up the various processors before the Linux kernel boots.

On boot, Linux mounts the boot partition as `/boot/firmware/`.

bootcode.bin

The bootloader, loaded by the SoC on boot. It performs some very basic setup, and then loads one of the `start*.elf` files.

The Raspberry Pi 4 and 5 do not use `bootcode.bin`. It has been replaced by boot code in the [onboard EEPROM](#).

start*.elf

Binary firmware blobs loaded onto the VideoCore GPU in the SoC, which then take over the boot process.

start.elf
the basic firmware.

start_x.elf
includes additional codecs.

start_db.elf
used for debugging.

start_cd.elf
a cut-down version of the firmware that removes support for hardware blocks such as codecs and 3D as well as debug logging support. It also imposes initial frame buffer limitations.

`start4.elf`, `start4x.elf`, `start4db.elf` and `start4cd.elf` are equivalent firmware files specific to the Raspberry Pi 4-series (Model 4B, Raspberry Pi 400, Compute Module 4 and Compute Module 4S).

For more information on how to use these files, see the `config.txt` [documentation](#).

The Raspberry Pi 5 does not use `elf` files. The firmware is self-contained within the bootloader EEPROM.

fixup*.dat

Linker files found in matched pairs with the `start*.elf` files listed in the previous section.

cmdline.txt

The **kernel command line** passed into the kernel at boot.

config.txt

Contains many configuration parameters for setting up the Raspberry Pi. For more information, see the [config.txt documentation](#).

IMPORTANT

Raspberry Pi 5 requires a non-empty config.txt file in the boot partition.

issue.txt

Text-based housekeeping information containing the date and git commit ID of the distribution.

initramfs*

Contents of the initial ramdisk. This loads a temporary root file system into memory before the real root file system can be mounted.

Raspberry Pi OS *Bookworm* includes an initramfs file by default. To enable the initial ramdisk, configure it in config.txt with the auto_initramfs keyword.

Raspberry Pi OS *Trixie* includes the auto_initramfs keyword by default.

ssh or ssh.txt

When this file is present, enables SSH at boot. Unless you've enabled SSH in Imager, this is disabled by default.

The contents do not matter: even an empty file enables SSH.

Device Tree blob files (*.dtb)

Device tree blob files contain the hardware definitions of the various models of Raspberry Pi. These files set up the kernel at boot **based on the detected Raspberry Pi model**.

Kernel files (*.img)

Various **kernel** image files that correspond to Raspberry Pi models:

Filename	Processor	Raspberry Pi model	Notes
kernel.img	BCM2835	Pi Zero, Pi 1, CM1	
kernel7.img	BCM2836, BCM2837	Pi Zero 2 W, Pi 2, Pi 3, CM3, Pi 3+, CM3+	Early revisions of Raspberry Pi 2 used BCM2836
kernel71.img	BCM2711	Pi 4, CM4, CM4S, Pi 400	Large Physical Address Extension (LPAE)
kernel8.img	BCM2837, BCM2711, BCM2712	Pi Zero 2 W, Pi 2 (later revisions), Pi 3, CM3, Pi 3+, CM3+, Pi 4, CM4, CM4S, Pi 400, CM5, Pi 5, Pi 500, Pi 500+	64-bit kernel . Earlier revisions of Raspberry Pi 2 (with BCM2836) do not support 64-bit kernels.
kernel_2712.img	BCM2712	Pi 5, CM5, Pi 500, Pi 500+	Raspberry Pi 5-optimized 64-bit kernel .

NOTE

lscpu reports a CPU architecture of armv71 for systems running a 32-bit kernel, and aarch64 for systems running a 64-bit kernel. The 1 in the armv71 case refers to little-endian CPU architecture, not LPAE as is indicated by the 1 in the kernel71.img filename.

overlays folder

Contains Device Tree overlays. These are used to configure various hardware devices, such as third-party sound boards. Entries in `config.txt` select these overlays. For more information, see [Device Trees, overlays and parameters](#).

Device Trees, overlays, and parameters

Raspberry Pi kernels and firmware use a Device Tree (DT) to describe hardware. These Device Trees may include DT parameters that control onboard features. DT overlays allow optional external hardware to be described and configured, and they also support parameters for more control.

The firmware loader (`start.elf` and its variants) is responsible for loading the DTB (Device Tree Blob – a machine-readable DT file). It chooses which one to load based on the board revision number, and makes modifications to further tailor it. This runtime customisation avoids the need for many DTBs with only minor differences.

User-provided parameters in `config.txt` are scanned, along with any overlays and their parameters, which are then applied. The loader examines the result to learn (for example) which UART, if any, is to be used for the console. Finally it launches the kernel, passing a pointer to the merged DTB.

Device Trees

A Device Tree (DT) is a description of the hardware in a system.

It includes the name of the base CPU, its memory configuration, and any peripherals (internal and external). It is not used to describe the software, although by listing the hardware modules it can cause driver modules to be loaded.

NOTE

It helps to remember that DTs are supposed to be OS-neutral, so anything which is Linux-specific shouldn't be there.

A Device Tree represents the hardware configuration as a hierarchy of nodes. Each node may contain properties and subnodes. Properties are named arrays of bytes, which may contain strings, numbers (big-endian), arbitrary sequences of bytes, and any combination thereof. By analogy to a filesystem, nodes are directories and properties are files. The locations of nodes and properties within the tree can be described using a path, with slashes as separators and a single slash (/) to indicate the root.

Basic DTS syntax

Device Trees are usually written in a textual form known as Device Tree Source (DTS), and are stored in files with a `.dts` suffix. DTS syntax is C-like, with braces for grouping and semicolons at the end of each line. Note that DTS requires semicolons after closing braces: think of C structs rather than functions. The compiled binary format is referred to as Flattened Device Tree (FDT) or Device Tree Blob (DTB), and is stored in `.dtb` files.

The following is a simple tree in the `.dts` format:

```
/dts-v1;
/include/ "common.dtsi";

/ {
    node1 {
        a-string-property = "A string";
        a-string-list-property = "first string", "second string";
        a-byte-data-property = [0x01 0x23 0x34 0x56];
        cousin: child-node1 {
            first-child-property;
            second-child-property = <1>;
            a-string-property = "Hello, world";
        };
        child-node2 {
        };
    };
    node2 {
        an-empty-property;
        a-cell-property = <1 2 3 4>; /* each number (cell) is a uint32 */
        child-node1 {
            my-cousin = <&cousin>;
        };
    };
};
```

```
};
};

/node2 {
    another-property-for-node2;
};
```

This tree contains:

- a required header: `/dts-v1/`
- The inclusion of another DTS file, conventionally named `*.dtsi` and analogous to a `.h` header file in C
- a single root node: `/`
- a couple of child nodes: `node1` and `node2`
- some children for `node1`: `child-node1` and `child-node2`
- a label (`cousin`) and a reference to that label (`&cousin`)
- several properties scattered through the tree
- a repeated node (`/node2`)

Properties are simple key-value pairs where the value can either be empty or contain an arbitrary byte stream. While data types are not encoded in the data structure, there are a few fundamental data representations that can be expressed in a Device Tree source file.

Text strings (NUL-terminated) are indicated with double quotes:

```
string-property = "a string";
```

Cells are 32-bit unsigned integers delimited by angle brackets:

```
cell-property = <0xbeef 123 0xabcd1234>;
```

Arbitrary byte data is delimited with square brackets, and entered in hex:

```
binary-property = [01 23 45 67 89 ab cd ef];
```

Data of differing representations can be concatenated using a comma:

```
mixed-property = "a string", [01 23 45 67], <0x12345678>;
```

Commas are also used to create lists of strings:

```
string-list = "red fish", "blue fish";
```

An aside about `/include/`

The `/include/` directive results in simple textual inclusion, much like C's `#include` directive, but a feature of the Device Tree compiler leads to different usage patterns. Given that nodes are named, potentially with absolute paths, it is possible for the same node to appear twice in a DTS file (and its inclusions). When this happens, the nodes and properties are combined, interleaving and overwriting properties as required (later values override earlier ones).

In the example above, the second appearance of `/node2` causes a new property to be added to the original:

```
/node2 {
    an-empty-property;
    a-cell-property = <1 2 3 4>; /* each number (cell) is a uint32 */
    another-property-for-node2;
    child-node1 {
        my-cousin = <&cousin>;
    };
};
```

It is therefore possible for one `.dtsi` to overwrite, or provide defaults for, multiple places in a tree.

Labels and references

It is often necessary for one part of the tree to refer to another, and there are four ways to do this:

Path strings

Similar to filesystem paths, for example, `/soc/i2s@7e203000` is the full path to the I2S device in BCM2835 and BCM2836. The standard APIs don't create paths to properties like `/soc/i2s@7e203000/status`: instead, you first find a node, then choose properties of that node.

Phandles

A unique 32-bit integer assigned to a node in its `phandle` property. For historical reasons, you may also see a redundant, matching `linux,phandle`. Phandles are numbered sequentially, starting from 1; 0 is not a valid phandle. They are usually allocated by the DT compiler when it encounters a reference to a node in an integer context, usually in the form of a label. References to nodes using phandles are simply encoded as the corresponding integer (cell) values; there is no markup to indicate that they should be interpreted as phandles, as that is application-defined.

Labels

Just as a label in C gives a name to a place in the code, a DT label assigns a name to a node in the hierarchy. The compiler takes references to labels and converts them into paths when used in string context (`&node`) and phandles in integer context (`<&node>`); the original labels do not appear in the compiled output. Note that labels contain no structure; they are just tokens in a flat, global namespace.

Aliases

Similar to labels, except that they do appear in the FDT output as a form of index. They are stored as properties of the `/aliases` node, with each property mapping an alias name to a path string. Although the aliases node appears in the source, the path strings usually appear as references to labels (`&node`), rather than being written out in full. DT APIs that resolve a path string to a node typically look at the first character of the path, treating paths that do not start with a slash as aliases that must first be converted to a path using the `/aliases` table.

Device Tree semantics

How to construct a Device Tree, and how best to use it to capture the configuration of some hardware, is a large and complex subject. There are many resources available, some of which are listed below, but several points deserve highlighting:

- `compatible` properties are the link between the hardware description and the driver software. When an OS encounters a node with a `compatible` property, it looks it up in its database of device drivers to find the best match. In Linux, this usually results in the driver module being automatically loaded, provided it has been appropriately labelled and not blacklisted.
- The `status` property indicates whether a device is enabled or disabled. If the `status` is `ok`, `okay` or `absent`, then the device is enabled. Otherwise, `status` should be `disabled`, so that the device is disabled. It can be useful to place devices in a `.dtsi` file with the status set to `disabled`. A derived configuration can then include that `.dtsi` and set the status for the devices which are needed to `okay`.

Device Tree overlays

A modern System on a Chip (SoC) is a very complicated device; a complete Device Tree could be hundreds of lines long. Taking that one step further and placing the SoC on a board with other components only makes matters more complicated. To keep that manageable, particularly if there are related devices which share components, it makes sense to put the common elements in `.dtsi` files, to be included from possibly multiple `.dts` files.

When a system like Raspberry Pi also supports optional plug-in accessories such as HATs, the problem grows. Ultimately, each possible configuration requires a Device Tree to describe it, but once you factor in all the different base models and the large number of available accessories, the number of combinations starts to multiply rapidly.

What is needed is a way to describe these optional components using a partial Device Tree, and then to be able to build a complete tree by taking a base DT and adding a number of optional elements. You can do this, and these optional elements are called "overlays".

Unless you want to learn how to write overlays for Raspberry Pis, you might prefer to skip on to [Use Device Trees](#).

Fragments

A DT overlay comprises a number of fragments, each of which targets one node and its subnodes. Although the concept sounds simple enough, the syntax seems rather strange at first:

```
// Enable the i2s interface
/dts-v1/;
/plugin/;

/ {
    compatible = "brcm,bcm2835";

    fragment@0 {
        target = <&i2s>;
        __overlay__ {
            status = "okay";
            test_ref = <&test_label>;
            test_label: test_subnode {
                dummy;
            };
        };
    };
};
```

The `compatible` string identifies this as being for BCM2835, which is the base architecture for the Raspberry Pi SoCs; if the overlay makes use of features of a Raspberry Pi 4 then `brcm,bcm2711` is the correct value to use, otherwise `brcm,bcm2835` can be used for all Raspberry Pi overlays. Then comes the first (and in this case only) fragment. Fragments should be numbered sequentially from zero. Failure to adhere to this may cause some or all of your fragments to be missed.

Each fragment consists of two parts: a `target` property, identifying the node to apply the overlay to; and the `__overlay__` itself, the body of which is added to the target node. The example above can be interpreted as if it were written like this:

```
/dts-v1/;
/plugin/;

/ {
    compatible = "brcm,bcm2835";
};

&i2s {
    status = "okay";
    test_ref = <&test_label>;
    test_label: test_subnode {
        dummy;
    };
};
```

With a sufficiently new version of `dtc` you can write the example exactly as above and get identical output, but some homegrown tools don't understand this format yet. Any overlay that you might want to see included in the standard Raspberry Pi OS kernel should be written in the old format for now.

The effect of merging that overlay with a standard Raspberry Pi base Device Tree (for example, `bcm2708-rpi-b-plus.dtb`), provided the overlay is loaded afterwards, would be to enable the I2S interface by changing its status to `okay`. But if you try to compile this overlay using:

```
$ dtc -I dts -O dtb -o 2nd.dtbo 2nd-overlay.dts
```

...you will get an error:

```
Label or path i2s not found
```

This shouldn't be too unexpected, since there is no reference to the base `.dtb` or `.dts` file to allow the compiler to find the `i2s` label.

Trying again, this time using the original example and adding the `-@` option to allow unresolved references (and `-Hepapr` to remove some clutter):

```
$ dtc -@ -Hepapr -I dts -O dtb -o 1st.dtbo 1st-overlay.dts
```

If `dtc` returns an error about the third line, it doesn't have the extensions required for overlay work. Run `sudo apt install device-tree-compiler` and try again – this time, compilation should complete successfully. Note that a suitable compiler is also available in the kernel tree as `scripts/dtc/dtc`, built when the dtbs make target is used:

```
$ make ARCH=arm dtbs
```

Dump the contents of the DTB file to see what the compiler has generated:

```
$ fdt dump 1st.dtbo
```

This should output something similar to the following:

```
/dts-v1/;
// magic:                0xd00df00d
// totalsize:            0x207 (519)
// off_dt_struct:        0x38
// off_dt_strings:        0x1c8
// off_mem_rsvmap:        0x28
// version:              17
// last_comp_version:     16
// boot_cpuid_phys:       0x0
// size_dt_strings:       0x3f
// size_dt_struct:        0x190

/ {
    compatible = "brcm,bcm2835";
    fragment@0 {
        target = <0xffffffff>;
        __overlay__ {
            status = "okay";
            test_ref = <0x00000001>;
            test_subnode {
                dummy;
                phandle = <0x00000001>;
            };
        };
    };
    __symbols__ {
        test_label = "/fragment@0/__overlay__/test_subnode";
    };
    __fixups__ {
        i2s = "/fragment@0:target:0";
    };
    __local_fixups__ {
        fragment@0 {
            __overlay__ {
                test_ref = <0x00000000>;
            };
        };
    };
};
```

After the verbose description of the file structure there is our fragment. But look carefully – where we wrote `&i2s` it now says `0xffffffff`, a clue that something strange has happened (older versions of `dtc` might say `0xdeadbeef` instead). The compiler has also added a `phandle` property containing a unique (to this overlay) small integer to indicate that the node has a label, and replaced all references to the label with the same small integer.

After the fragment there are three new nodes:

- `__symbols__` lists the labels used in the overlay (`test_label` here), and the path to the labelled node. This node is the key to how unresolved symbols are dealt with.
- `__fixups__` contains a list of properties mapping the names of unresolved symbols to lists of paths to cells within the fragments that need patching with the `phandle` of the target node, when that target has been located. In this case, the path is to the `0xffffffff` value of `target`, but fragments can contain other unresolved references which would require additional fixes.
- `__local_fixups__` holds the locations of any references to labels that exist within the overlay – the `test_ref` property. This is required because the program performing the merge will have to ensure that `phandle` numbers are sequential and unique.

Back in [section 1.3](#) it says that "the original labels do not appear in the compiled output", but this isn't true when using the `-@` switch. Instead, every label results in a property in the `__symbols__` node, mapping a label to a path, exactly like the `aliases` node. In fact, the mechanism is so similar that when resolving symbols, the Raspberry Pi loader will search the "aliases" node in the absence of a `__symbols__` node. This was useful at one time because providing sufficient aliases allowed very old versions of `dtc` to be used to build the base DTB files, but fortunately that is ancient history now.

Device Tree parameters

To avoid the need for lots of Device Tree overlays, and to reduce the need for users of peripherals to modify DTS files, the Raspberry Pi loader supports a new feature – Device Tree parameters. This permits small changes to the DT using named parameters, similar to the way kernel modules receive parameters from `modprobe` and the kernel command line. Parameters can be exposed by the base DTBs and by overlays, including HAT overlays.

Parameters are defined in the DTS by adding an `__overrides__` node to the root. It contains properties whose names are the chosen parameter names, and whose values are a sequence comprising a phandle (reference to a label) for the target node, and a string indicating the target property; string, integer (cell) and boolean properties are supported.

String parameters

String parameters are declared like this:

```
name = <&label>, "property";
```

where `label` and `property` are replaced by suitable values. String parameters can cause their target properties to grow, shrink, or be created.

Note that properties called `status` are treated specially; non-zero/true/yes/on values are converted to the string "okay", while zero/false/no/off becomes "disabled".

Integer parameters

Integer parameters are declared like this:

```
name = <&label>, "property.offset"; // 8-bit
name = <&label>, "property;offset"; // 16-bit
name = <&label>, "property:offset"; // 32-bit
name = <&label>, "property#offset"; // 64-bit
```

Here, `label`, `property` and `offset` are replaced by suitable values; the `offset` is specified in bytes relative to the start of the property (in decimal by default), and the preceding separator dictates the size of the parameter. In a change from earlier implementations, integer parameters may refer to non-existent properties or to offsets beyond the end of an existing property.

Boolean parameters

Device Tree encodes boolean values as zero-length properties; if present then the property is true, otherwise it is false. They are defined like this:

```
boolean_property; // Set 'boolean_property' to true
```

A property is assigned the value `false` by not defining it. Boolean parameters are declared like this, replacing the `label` and `property` placeholders with suitable values:

```
name = <&label>, "property?";
```

Inverted booleans invert the input value before applying it in the same way as a regular boolean; they are declared similarly, but use `!` to indicate the inversion:

```
name = <&label>, "<property>!";
```

Boolean parameters can cause properties to be created or deleted, but they can't delete a property that already exists in the base DTB.

Byte string parameters

Byte string properties are arbitrary sequences of bytes, for example, MAC addresses. They accept strings of hexadecimal bytes, with or without colons between the bytes.

```
mac_address = <ethernet0>,"local_mac_address[";
```

The [was chosen to match the DT syntax for declaring a byte string:

```
local_mac_address = [aa bb cc dd ee ff];
```

Parameters with multiple targets

There are some situations where it is convenient to be able to set the same value in multiple locations within the Device Tree. Rather than the ungainly approach of creating multiple parameters, it is possible to add multiple targets to a single parameter by concatenating them, like this:

```
_overrides_ {
    gpioin = <&w1>,"gpios:4",
            <&w1_pins>,"brcm,pins:0";
    ...
};
```

(example taken from the w1-gpio overlay)

NOTE

It is even possible to target properties of different types with a single parameter. You could reasonably connect an "enable" parameter to a status string, cells containing zero or one, and a proper boolean property.

Literal assignments

The DT parameter mechanism allows multiple targets to be patched from the same parameter, but the utility is limited by the fact that the same value has to be written to all locations (except for format conversion and the negation available from inverted booleans). The addition of embedded literal assignments allows a parameter to write arbitrary values, regardless of the parameter value supplied by the user.

Assignments appear at the end of a declaration, and are indicated by a =:

```
str_val = <&target>,"strprop=value";           // 1
int_val = <&target>,"intprop:0=42"              // 2
int_val2 = <&target>,"intprop:0=",<42>;         // 3
bytes   = <&target>,"bytestr[=b8:27:eb:01:23:45"; // 4
```

Lines 1, 2 and 4 are fairly obvious, but line 3 is more interesting because the value appears as an integer (cell) value. The DT compiler evaluates integer expressions at compile time, which might be convenient (particularly if macro values are used), but the cell can also contain a reference to a label:

```
// Force an LED to use a GPIO on the internal GPIO controller.
exp_led = <&led1>,"gpios:0=",<&gpio>,
          <&led1>,"gpios:4";
```

When the overlay is applied, the label will be resolved against the base DTB in the usual way. It is a good idea to split multi-part parameters over multiple lines like this to make them easier to read – something that becomes more necessary with the addition of cell value assignments.

Bear in mind that parameters do nothing unless they are applied – a default value in a lookup table is ignored unless the parameter name is used without assigning a value.

Lookup tables

Lookup tables allow parameter input values to be transformed before they are used. They act as associative arrays, rather like switch/case statements:

```
phonetic = <node>,"letter{a=alpha,b=bravo,c=charlie,d,e='tango uniform'}";
bus      = <fragment>,"target:0{0=",<i2c0>,"1=",<i2c1>,"}";
```


A key with no `=value` means to use the key as the value, an `=` with no key before it is the default value in the case of no match, and starting or ending the list with a comma (or an empty `key=value` pair anywhere) indicates that the unmatched input value should be used unaltered; otherwise, not finding a match is an error.

NOTE

The comma separator within the table string after a cell integer value is implicit – adding one explicitly creates an empty pair (see above).

NOTE

As lookup tables operate on input values and literal assignments ignore them, it's not possible to combine the two; any characters after the closing `}` in the lookup declaration are treated as an error.

Overlay/fragment parameters

The DT parameter mechanism as described has a number of limitations, including the lack of an easy way to create arrays of integers, and the inability to create new nodes. One way to overcome some of these limitations is to conditionally include or exclude certain fragments.

A fragment can be excluded from the final merge process (disabled) by renaming the `__overlay__` node to `__dormant__`. The parameter declaration syntax has been extended to allow the otherwise illegal zero target phandle to indicate that the following string contains operations at fragment or overlay scope. So far, four operations have been implemented:

```
+<n>    // Enable fragment <n>
-<n>    // Disable fragment <n>
=<n>    // Enable fragment <n> if the assigned parameter value is true, otherwise disable it
!<n>    // Enable fragment <n> if the assigned parameter value is false, otherwise disable it
```

Examples:

```
just_one    = <0>, "+1-2"; // Enable 1, disable 2
conditional = <0>, "=3!4"; // Enable 3, disable 4 if value is true,
                          // otherwise disable 3, enable 4.
```

The `i2c-rtc` overlay uses this technique.

Special properties

A few property names, when targeted by a parameter, get special handling. One you may have noticed already – `status` – will convert a boolean to either okay for true or disabled for false.

Assigning to the `bootargs` property appends to it rather than overwriting it – this is how settings can be added to the kernel command line.

The `reg` property is used to specify device addresses – the location of a memory-mapped hardware block, the address on an I2C bus, etc. The names of child nodes should be qualified with their addresses in hexadecimal, using `@` as a separator:

```
bmp280@76 {
    reg = <0x77>;
    ...
};
```

When assigning to the `reg` property, the address portion of the parent node name will be replaced with the assigned value. This can be used to prevent a node name clash when using the same overlay multiple times – a technique used by the `i2c-gpio` overlay.

The `name` property is a pseudo-property – it shouldn't appear in a DT, but assigning to it causes the name of its parent node to be changed to the assigned value. Like the `reg` property, this can be used to give nodes unique names.

The overlay map file

The introduction of the Raspberry Pi 4, built around the BCM2711 SoC, brought with it many changes; some of these changes are additional interfaces, and some are modifications to (or removals of) existing interfaces. There are new overlays intended specifically for the Raspberry Pi 4 that don't make sense on older hardware, for example, overlays that enable the new SPI, I2C and UART interfaces, but other overlays don't apply correctly even though they control features that are still relevant on the new device.

There is therefore a need for a method of tailoring an overlay to multiple platforms with differing hardware. Supporting them all in a single .dtbo file would require heavy use of hidden ("dormant") fragments and a switch to an on-demand symbol resolution mechanism so that a missing symbol that isn't needed doesn't cause a failure. A simpler solution is to add a facility to map an overlay name to one of several implementation files depending on the current platform.

The overlay map is a file that gets loaded by the firmware at bootup. It is written in DTS source format – `overlay_map.dts`, compiled to `overlay_map.dtb` and stored in the overlays directory.

This is an extract from the current map file (see the [full version](#)):

```
/ {
    disable-bt {
        bcm2835;
        bcm2711;
        bcm2712 = "disable-bt-pi5";
    };

    disable-bt-pi5 {
        bcm2712;
    };

    uart5 {
        bcm2711;
    };

    pi3-disable-bt {
        renamed = "disable-bt";
    };

    lirc-rpi {
        deprecated = "use gpio-ir";
    };
};
```

Each node has the name of an overlay that requires special handling. The properties of each node are either platform names or one of a small number of special directives. The overlay map supports the following platform names:

- `bcm2835` for all Raspberry Pis built around the BCM2835, BCM2836, BCM2837, and RP3A0 SoCs
- `bcm2711` for Raspberry Pi 4B, CM4, CM4S, and Pi 400
- `bcm2712` for Raspberry Pi 5, CM5, Pi 500, and Pi 500+

A platform name with no value (an empty property) indicates that the current overlay is compatible with the platform; for example, `uart5` is compatible with the `bcm2711` platform. A non-empty value for a platform is the name of an alternative overlay to use in place of the requested one; asking for `disable-bt` on BCM2712 results in `disable-bt-pi5` being loaded instead. Any platform not included in an overlay's node is not compatible with that overlay. Any overlay not mentioned in the map is assumed to be compatible with all platforms.

The second example node – `disable-bt-pi5` – could be inferred from the content of `disable-bt`, but that intelligence goes into the construction of the file, not its interpretation.

The `uart5` overlay only makes sense on BCM2711.

In the event that a platform is not listed for an overlay, one of the special directives may apply:

- The `renamed` directive indicates the new name of the overlay (which should be largely compatible with the original), but also logs a warning about the rename.

- The deprecated directive contains a brief explanatory error message which will be logged after the common prefix overlay '...' is deprecated.

Chaining renames and platform-specific implementations is possible, but be careful to avoid loops!

Remember: only exceptions need to be listed – the absence of a node for an overlay means that the default file should be used for all platforms.

Accessing diagnostic messages from the firmware is covered in [Debugging](#).

The dtoverlay and dtmerge utilities have been extended to support the map file:

- dtmerge extracts the platform name from the compatible string in the base DTB.
- dtoverlay reads the compatible string from the live Device Tree at /proc/device-tree, but you can use the -p option to supply an alternate platform name (useful for dry runs on a different platform).

They both send errors, warnings and any debug output to STDERR.

Examples

Here are some examples of different types of properties, with parameters to modify them:

```
/ {
    fragment@0 {
        target-path = "/";
        __overlay__ {

            test: test_node {
                string = "hello";
                status = "disabled";
                bytes = /bits/ 8 <0x67 0x89>;
                u16s = /bits/ 16 <0xabcd 0xef01>;
                u32s = /bits/ 32 <0xfedcba98 0x76543210>;
                u64s = /bits/ 64 < 0xaaaaa5a5a5a5a555 0x0000111122223333>;
                bool1; // Defaults to true
                // bool2 defaults to false
                mac = [01 23 45 67 89 ab];
                spi = <&spi0>;
            };
        };
    };

    fragment@1 {
        target-path = "/";
        __overlay__ {
            frag1;
        };
    };

    fragment@2 {
        target-path = "/";
        __dormant__ {
            frag2;
        };
    };

    __overrides__ {
        string = <&test>,"string";
        enable = <&test>,"status";
        byte_0 = <&test>,"bytes.0";
        byte_1 = <&test>,"bytes.1";
        u16_0 = <&test>,"u16s;0";
        u16_1 = <&test>,"u16s;2";
        u32_0 = <&test>,"u32s;0";
        u32_1 = <&test>,"u32s;4";
        u64_0 = <&test>,"u64s#0";
        u64_1 = <&test>,"u64s#8";
        bool1 = <&test>,"bool1!";
        bool2 = <&test>,"bool2?";
        entofr = <&test>,"english",
        <&test>,"french{hello=bonjour,goodbye='au revoir',week
end}";
        pi_mac = <&test>,"mac[{1=b8273bfedcba,2=b8273b987654}]";
        spi0 = <&test>,"spi:0[0=",<&spi0>,"1=",<&spi1>,"2=",<&spi2>;

        only1 = <0>,"+1-2";
        only2 = <0>,"-1+2";
        enable1 = <0>,"=1";
        disable2 = <0>,"!2";
    };
};
```

For further examples, a large collection of overlay source files is hosted in the [Raspberry Pi Linux GitHub repository](#).

Export labels

The overlay handling in the firmware, and the run-time overlay application using the `dtoverlay` utility, treat labels defined in an overlay as being private to that overlay. This avoids the need to invent globally unique names for labels (which keeps them short), and it allows the same overlay to be used multiple times without clashing (provided some tricks are used – see [Special properties](#)).

Sometimes it is very useful to be able to create a label with one overlay and use it from another. Firmware released since 14th February 2020 has the ability to declare some labels as being global – the `__exports__` node:

```
...
public: ...

__exports__ {
    public; // Export the label 'public' to the base DT
};
};
```

When this overlay is applied, the loader strips out all symbols except those that have been exported, in this case `public`, and rewrites the path to make it relative to the target of the fragment containing the label. Overlays loaded after this one can then refer to `&public`.

Overlay application order

Under most circumstances it shouldn't matter in which order the fragments are applied, but for overlays that patch themselves (where the target of a fragment is a label in the overlay, known as an intra-overlay fragment) it becomes important. In older firmware, fragments are applied strictly in order, top to bottom. With firmware released since 14th February 2020, fragments are applied in two passes:

- First the fragments that target other fragments are applied and hidden.
- Then the regular fragments are applied.

This split is particularly important for runtime overlays, since the first step occurs in the `dtoverlay` utility, and the second is performed by the kernel (which can't handle intra-overlay fragments).

Using Device Trees on Raspberry Pi

DTBs, overlays and `config.txt`

On a Raspberry Pi it is the job of the loader (one of the `start.elf` images) to combine overlays with an appropriate base device tree, and then to pass a fully resolved Device Tree to the kernel. The base Device Trees are located alongside `start.elf` in the FAT partition (`/boot/firmware/` from Linux), named `bcm2711-rpi-4-b.dtb`, `bcm2710-rpi-3-b-plus.dtb`, etc. Note that some models (3A+, A, A+) will use the "b" equivalents (3B+, B, B+), respectively. This selection is automatic, and allows the same SD card image to be used in a variety of devices.

NOTE

DT and ATAGs are mutually exclusive, and passing a DT blob to a kernel that doesn't understand it will cause a boot failure. The firmware will always try to load the DT and pass it to the kernel, since all kernels since `rpi-4.4.y` will not function without a DTB. You can override this by adding `device_tree=` in `config.txt`, which forces the use of ATAGs, which can be useful for simple bare-metal kernels.

The loader now supports builds using `bcm2835_defconfig`, which selects the upstreamed BCM2835 support. This configuration will cause `bcm2835-rpi-b.dtb` and `bcm2835-rpi-b-plus.dtb` to be built. If these files are copied with the kernel, then the loader will attempt to load one of those DTBs by default.

In order to manage Device Tree and overlays, the loader supports a number of `config.txt` directives:

```
dtoverlay=acme-board
dtparam=foo=bar,level=42
```

This will cause the loader to look for `overlays/acme-board.dtbo` in the firmware partition, which Raspberry Pi OS mounts on `/boot/firmware/`. It will then search for parameters `foo` and `level`, and assign the indicated values to them.

The loader will also search for an attached HAT with a programmed EEPROM, and load the supporting overlay from there – either directly or by name from the "overlays" directory; this happens without any user intervention.

There are multiple ways to tell that the kernel is using Device Tree:

- The "Machine model:" kernel message during bootup has a board-specific value such as "Raspberry Pi 2 Model B", rather than "BCM2709".
- `/proc/device-tree` exists, and contains subdirectories and files that exactly mirror the nodes and properties of the DT.

With a Device Tree, the kernel will automatically search for and load modules that support the indicated enabled devices. As a result, by creating an appropriate DT overlay for a device you save users of the device from having to edit `/etc/modules`; all of the configuration goes in `config.txt`, and in the case of a HAT, even that step is unnecessary. Note, however, that layered modules such as `i2c-dev` still need to be loaded explicitly.

The flipside is that because platform devices don't get created unless requested by the DTB, it should no longer be necessary to blacklist modules that used to be loaded as a result of platform devices defined in the board support code. In fact, current Raspberry Pi OS images ship with no blacklist files (except for some WLAN devices where multiple drivers are available).

DT parameters

As described above, DT parameters are a convenient way to make small changes to a device's configuration. The current base DTBs support parameters for enabling and controlling the onboard audio, I2C, I2S and SPI interfaces without using dedicated overlays. In use, parameters look like this:

```
dtparam=audio=on,i2c_arm=on,i2c_arm_baudrate=400000,spi=on
```

NOTE

Multiple assignments can be placed on the same line, but ensure you don't exceed the 80-character limit.

If you have an overlay that defines some parameters, they can be specified either on subsequent lines like this:

```
dtoverlay=lirc-rpi
dtparam=gpio_out_pin=16
dtparam=gpio_in_pin=17
dtparam=gpio_in_pull=down
```

...or appended to the overlay line like this:

```
dtoverlay=lirc-rpi,gpio_out_pin=16,gpio_in_pin=17,gpio_in_pull=down
```

Overlay parameters are only in scope until the next overlay is loaded. In the event of a parameter with the same name being exported by both the overlay and the base, the parameter in the overlay takes precedence; it's recommended that you avoid doing this. To expose the parameter exported by the base DTB instead, end the current overlay scope using:

```
dtoverlay=
```

Board-specific labels and parameters

Raspberry Pi boards have two I2C interfaces. These are nominally split: one for the Arm CPU, and one for the VideoCore GPU. On almost all models, `i2c1` belongs to the CPU and `i2c0` to the GPU, where it is used to control the camera and read the HAT EEPROM. However, there are two early revisions of the Model B that have those roles reversed.

To make it possible to use one set of overlays and parameters with all Raspberry Pis, the firmware creates some board-specific DT parameters. These are:

```
i2c/i2c_arm
i2c_vc
i2c_baudrate/i2c_arm_baudrate
i2c_vc_baudrate
```

These are aliases for `i2c0`, `i2c1`, `i2c0_baudrate`, and `i2c1_baudrate`. It is recommended that you only use `i2c_vc` and `i2c_vc_baudrate` if you really need to, for example, if you are programming a HAT EEPROM (which is better done using a software I2C bus using the `i2c-gpio` overlay). Enabling `i2c_vc` can stop the Raspberry Pi Camera or Raspberry Pi Touch Display functioning correctly.

For people writing overlays, the same aliasing has been applied to the labels on the I2C DT nodes. Thus, you should write:

```
fragment@0 {
    target = <&i2c_arm>;
    __overlay__ {
        status = "okay";
    };
};
```

Any overlays using the numeric variants will be modified to use the new aliases.

HATs and Device Tree

A Raspberry Pi HAT is an add-on board with an embedded EEPROM designed for a Raspberry Pi with a 40-pin header. The EEPROM includes any DT overlay required to enable the board (or the name of an overlay to load from the filing system), and this overlay can also expose parameters.

The HAT overlay is automatically loaded by the firmware after the base DTB, so its parameters are accessible until any other overlays are loaded, or until the overlay scope is ended using `dtoverlay=`. If for some reason you want to suppress the loading of the HAT overlay, put `dtoverlay=` before any other `dtoverlay` or `dtparam` directive.

Dynamic Device Tree

As of Linux 4.4, Raspberry Pi kernels support the dynamic loading of overlays and parameters. Compatible kernels manage a stack of overlays that are applied on top of the base DTB. Changes are immediately reflected in `/proc/device-tree` and can cause modules to be loaded and platform devices to be created and destroyed.

The use of the word "stack" above is important – overlays can only be added and removed at the top of the stack; changing something further down the stack requires that anything on top of it must first be removed.

There are some new commands for managing overlays:

The `dtoverlay` command

`dtoverlay` is a command line utility that loads and removes overlays while the system is running, as well as listing the available overlays and displaying their help information.

Use `dtoverlay -h` to get usage information:

```
Usage:
dtoverlay <overlay> [<param>=<val>...]
                                Add an overlay (with parameters)
dtoverlay -D [<idx>]           Dry-run (prepare overlay, but don't apply -
                                save it as dry-run.dtbo)
dtoverlay -r [<overlay>]       Remove an overlay (by name, index or the last)
dtoverlay -R [<overlay>]       Remove from an overlay (by name, index or all)
dtoverlay -l                   List active overlays/params
dtoverlay -a                   List all overlays (marking the active)
dtoverlay -h                   Show this usage message
dtoverlay -h <overlay>        Display help on an overlay
dtoverlay -h <overlay> <param>.. Or its parameters
                                where <overlay> is the name of an overlay or 'dtparam' for dtparams
Options applicable to most variants:
-d <dir>      Specify an alternate location for the overlays
```

```
(defaults to /boot/firmware/overlays or /flash/overlays)
-v          Verbose operation
```

Unlike the `config.txt` equivalent, all parameters to an overlay must be included in the same command line – the `dtparam` command is only for parameters of the base DTB.

Command variants that change kernel state (adding and removing things) require root privilege, so you may need to prefix the command with `sudo`. Only overlays and parameters applied at run-time can be unloaded – an overlay or parameter applied by the firmware becomes "baked in" such that it won't be listed by `dtoverlay` and can't be removed.

The `dtparam` command

`dtparam` creates and loads an overlay that has largely the same effect as using a `dtparam` directive in `config.txt`. In usage it is largely equivalent to `dtoverlay` with an overlay name of `-`, but there are a few differences: `dtparam` will list the help information for all known parameters of the base DTB. Help on the `dtparam` command is still available using `dtparam -h`. When indicating a parameter for removal, only index numbers can be used (not names). Not all Linux subsystems respond to the addition of devices at runtime – I2C, SPI and sound devices work, but some won't.

Guidelines for writing runtime-capable overlays

The creation or deletion of a device object is triggered by a node being added or removed, or by the status of a node changing from disabled to enabled or vice versa. The absence of a "status" property means the node is enabled.

Don't create a node within a fragment that will overwrite an existing node in the base DTB – the kernel will rename the new node to make it unique. If you want to change the properties of an existing node, create a fragment that targets it.

ALSA doesn't prevent its codecs and other components from being unloaded while they are in use. Removing an overlay can cause a kernel exception if it deletes a codec that is still being used by a sound card. Experimentation found that devices are deleted in the reverse of fragment order in the overlay, so placing the node for the card after the nodes for the components allows an orderly shutdown.

Caveats

The loading of overlays at runtime is a recent addition to the kernel, and at the time of writing there is no accepted way to do this from userspace. By hiding the details of this mechanism behind commands, users are insulated from changes in the event that a different kernel interface becomes standardised.

- Some overlays work better at run-time than others. Parts of the Device Tree are only used at boot time – changing them using an overlay will not have any effect.
- Applying or removing some overlays may cause unexpected behaviour, so it should be done with caution. This is one of the reasons it requires `sudo`.
- Unloading the overlay for an ALSA card can stall if something is actively using ALSA – the LXPanel volume slider plugin demonstrates this effect. To enable overlays for sound cards to be removed, the `lxpanelctl` utility has been given two new options – `alsastop` and `alsastart` – and these are called from the auxiliary scripts `dtoverlay-pre` and `dtoverlay-post` before and after overlays are loaded or unloaded, respectively.
- Removing an overlay will not cause a loaded module to be unloaded, but it may cause the reference count of some modules to drop to zero. Running `rmmmod -a` twice will cause unused modules to be unloaded.
- Overlays have to be removed in reverse order. The commands will allow you to remove an earlier one, but all the intermediate ones will be removed and re-applied, which may have unintended consequences.
- Only Device Tree nodes at the top level of the tree and children of a bus node will be probed. For nodes added at run-time there is the further limitation that the bus must register for notifications of the addition and removal of children. However,

there are exceptions that break this rule and cause confusion: the kernel explicitly scans the entire tree for some device types – clocks and interrupt controller being the two main ones – in order to (for clocks) initialise them early and/or (for interrupt controllers) in a particular order. This search mechanism only happens during booting and so doesn't work for nodes added by an overlay at run-time. It is therefore recommended for overlays to place fixed-clock nodes in the root of the tree unless it is guaranteed that the overlay will not be used at run-time.

Supported overlays and parameters

For a list of supported overlays and parameters, see the [README](#) file found alongside the overlay .dtbo files in /boot/firmware/overlays. It is kept up-to-date with additions and changes.

Firmware parameters

The firmware uses the special /chosen node to pass parameters between the bootloader and/or firmware and the operating system.

- Each property is stored as a 32-bit unsigned integer unless indicated otherwise.
- Numbers in device-tree are stored in binary and are big-endian.

Example shell command for reading a 32-bit unsigned integer property:

```
printf "%d" "$(od -t x1 | tr -d ' ')" "/proc/device-tree/chosen/bootloader/partition" -v -An -
```

overlay_prefix

(string) The overlay_prefix string selected by config.txt.

os_prefix

(string) The os_prefix string selected by config.txt.

rpi-boardrev-ext

The extended board revision code from [OTP row 33](#).

rpi-country-code

The country code used by [PiWiz](#). Keyboard models only.

rpi-duid

(string) Raspberry Pi 5 only. A string representation of the QR code on the PCB.

rpi-sdram-size-gbit

The physical SDRAM size in gigabits. Unlike /proc/meminfo, this value isn't affected by memory carveouts or the total_mem parameter. In scripts reporting device capabilities, use this value (if defined) instead of the **Memory size** field in the board revision code, which cannot represent non-power-of-two sizes such as 3 GB.

rpi-machine-id

(string) A 128-bit machine identifier represented as a 32-character hexadecimal string, generated from a hash of the 64-bit serial number and, on Raspberry Pi 4 and Raspberry Pi 5, the Ethernet MAC address provisioned at manufacture. This provides a stable, unique identity suitable for use in automated or provisioning systems.

rpi-min-boot-ver

The minimum bootloader manufacturing version required by the board's hardware, programmed into OTP during production. Before applying an update, rpi-eeeprom-update extracts the MFG_VER field from the EEPROM image and compares it against this value. If the image version is lower than the board's minimum, the update is rejected. The MFG_VER field embedded in an EEPROM image can be inspected with strings pieeprom.bin | grep 'MFG_VER:'. Raspberry Pi 4 and Raspberry Pi 5 only.

rpi-serial64

(string) A string representation of the 64-bit serial number. On flagship models since Raspberry Pi 5 this is same as the normal serial number (/proc/device-tree/serial-number). On earlier models the default serial number is still 32-bit but with newer firmware a 64-bit serial number is now available and is visible through this node.

Common bootloader properties /chosen/bootloader**boot-mode**

The boot-mode used to load the kernel. See the **BOOT_ORDER** documentation for a list of possible boot-mode values.

partition

The partition number used during boot. If a `boot.img` ramdisk is loaded then this refers to partition that the ramdisk was loaded from rather than the partition number within the ramdisk.

pm_rsts

The value of the PM_RSTS register during boot.

tryboot

Set to 1 if the tryboot flag was set at boot.

Boot variables /chosen/bootloader

Raspberry Pi 5 only.

arg1

The value of the user defined reboot argument from the previous boot. See **boot_arg1**

count

The value of the 8-bit `boot_count` variable when the OS was started. See **boot_count**

Power supply properties /chosen/power

Raspberry Pi 5 only.

max_current

The maximum current in mA that the power supply can supply. The firmware reports the value indicated by the USB-C, USB-PD or PoE interfaces. For bench power supplies (for example, connected to the GPIO header) define `PSU_MAX_CURRENT` in the bootloader configuration to indicate the power supply current capability.

power_reset

Raspberry Pi 5 only. A bit field indicating the reason why the PMIC was reset.

Bit	Reason
0	Over voltage
1	Under voltage
2	Over temperature
3	Enable signal
4	Watchdog

rpi_power_supply

(two 32-bit integers) The USB VID and Product VDO of the official Raspberry Pi 27 W power supply (if connected).

usb_max_current_enable

Zero if the USB port current limiter was set to the low-limit during boot; or non-zero if the high limit was enabled. The high level is automatically enabled if the power supply claims 5A max-current OR `usb_max_current_enable=1` is forced in `config.txt`

usb_over_current_detected

Non-zero if a USB over-current event occurred during USB boot.

usbpd_power_data_objects

(binary blob containing multiple 32-bit integers) The raw binary USB-PD objects (fixed supply only) received by the bootloader during USB-PD negotiation. To capture this for a bug report, run `hexdump -C /proc/device-tree/chosen/power/usbpd_power_data_objects`.

The format is defined by the **USB Power Delivery** specification.

BCM2711 and BCM2712 specific bootloader properties /chosen/bootloader

The following properties are specific to the BCM2711 and BCM2712 SPI EEPROM bootloaders.

build_timestamp

The UTC build time for the EEPROM bootloader, stored as a Unix epoch value. The build timestamp is embedded in the EEPROM binary and can be extracted with `strings pieeprom.bin | grep BUILD_TIMESTAMP`.

capabilities

This bit-field describes the features supported by the current bootloader. This may be used to check whether a feature (for example, USB boot) is supported before enabling it in the bootloader EEPROM config.

Bit	Feature
0	USB boot using the VLI USB host controller
1	Network boot
2	TRYBOOT_A_B mode
3	TRYBOOT
4	USB boot using the BCM2711 USB host controller
5	RAM disk - boot.img
6	NVMe boot
7	Secure Boot

update_timestamp

The UTC time when the EEPROM update was applied, stored as a Unix epoch value. This value is set by `rpi-eeeprom-update` and stored in the `.sig` file alongside the EEPROM image. The bootloader uses this to avoid applying stale updates during self-update.

signed

If Secure Boot is enabled, this bit-field will be non-zero. The individual bits indicate the current Secure Boot configuration.

Bit	Description
0	SIGNED_BOOT was defined in the EEPROM config file.
1	Reserved
2	The ROM development key has been revoked. See revoke_devkey .
3	The customer public key digest has been written to OTP. See program_pubkey .
4...31	Reserved

version

(string) The Git version string for the bootloader.

BCM2711 and BCM2712 USB boot properties /chosen/bootloader/usb

The following properties are defined if the system was booted from USB. These may be used to uniquely identify the USB boot device.

usb-version

The USB major protocol version (2 or 3).

route-string

The USB route-string identifier for the device as defined by the USB 3.0 specification.

root-hub-port-number

The root hub port number that the boot device is connected to – possibly via other USB hubs.

lun

The Logical Unit Number for the mass-storage device.

NVMEM nodes

The firmware provides read-only, in-memory copies of portions of the bootloader EEPROM via the **NVMEM** subsystem.

Each region appears as an NVMEM device under `/sys/bus/nvmem/devices/` with a named alias under `/sys/firmware/devicetree/base/aliases`.

Example shell script code for reading an NVMEM mode from **rpi-eeeprom-update**:

```
blconfig_alias="/sys/firmware/devicetree/base/aliases/blconfig"
blconfig_nvmem_path=""

if [ -f "${blconfig_alias}" ]; then
    blconfig_ofnode_path="/sys/firmware/devicetree/base/${strings "${blconfig_alias}" ""}
    blconfig_ofnode_link=$(find -L /sys/bus/nvmem -samefile "${blconfig_ofnode_path}" 2>/dev/null)
    if [ -e "${blconfig_ofnode_link}" ]; then
        blconfig_nvmem_path=$(dirname "${blconfig_ofnode_link}")
    fi
fi
```

blconfig

alias that refers to an NVMEM device that stores a copy of the bootloader EEPROM config file.

blpubkey

alias that points to an NVMEM device that stores a copy of the bootloader EEPROM public key (if defined) in binary format. The **rpi-bootloader-key-convert** utility can be used to convert the data into PEM format for use with OpenSSL.

For more information, see **secure-boot**.

Troubleshooting

Debugging

The loader will skip over missing overlays and bad parameters, but if there are serious errors, such as a missing or corrupt base DTB or a failed overlay merge, then the loader will fall back to a non-DT boot. If this happens, or if your settings don't behave as you expect, it is worth checking for warnings or errors from the loader:

```
$ sudo vclog --msg
```

Extra debugging can be enabled by adding `dtdebug=1` to `config.txt`.

You can create a human-readable representation of the current state of DT like this:

```
$ dtc -I fs /proc/device-tree
```

This can be useful to see the effect of merging overlays onto the underlying tree.

If kernel modules don't load as expected, check that they aren't blacklisted in `/etc/modprobe.d/raspi-blacklist.conf`; blacklisting shouldn't be necessary when using Device Tree. If that shows nothing untoward, you can also check that the module is exporting the correct aliases by searching `/lib/modules/<version>/modules.alias` for the `compatible` value. Otherwise, your driver is probably missing either:

```
.of_match_table = xxx_of_match,
```

or:

```
MODULE_DEVICE_TABLE(of, xxx_of_match);
```

Failing that, `depmod` has failed or the updated modules haven't been installed on the target filesystem.

Test overlays using `dtmerge`, `dtdiff` and `ovmerge`

Alongside the `dtoverlay` and `dtparam` commands is a utility for applying an overlay to a DTB – `dtmerge`. To use it you first need to obtain your base DTB, which can be obtained in one of two ways:

Generate it from the live DT state in `/proc/device-tree`:

```
$ dtc -I fs -O dtb -o base.dtb /proc/device-tree
```

This will include any overlays and parameters you have applied so far, either in `config.txt` or by loading them at runtime, which may or may not be what you want. Alternatively:

Copy it from the source DTBs in `/boot/firmware/`. This won't include overlays and parameters, but it also won't include any other modifications by the firmware. To allow testing of all overlays, the `dtmerge` utility will create some of the board-specific aliases ("i2c_arm", etc.), but this means that the result of a merge will include more differences from the original DTB than you might expect. The solution to this is to use `dtmerge` to make the copy:

```
$ dtmerge /boot/firmware/bcm2710-rpi-3-b.dtb base.dtb -
```

(the `-` indicates an absent overlay name).

You can now try applying an overlay or parameter:

```
$ dtmerge base.dtb merged.dtb - sd_oversclock=62
$ dtdiff base.dtb merged.dtb
```

which will return:

```
--- /dev/fd/63  2016-05-16 14:48:26.396024813 +0100
+++ /dev/fd/62  2016-05-16 14:48:26.396024813 +0100
@@ -594,7 +594,7 @@
    };

    sdhost@7e202000 {
-       brcm,oversclock-50 = <0x0>;
+       brcm,oversclock-50 = <0x3e>;
        brcm,pio-limit = <0x1>;
        bus-width = <0x4>;
        clocks = <0x8>;
```

You can also compare different overlays or parameters.

```
$ dtmerge base.dtb merged1.dtb /boot/firmware/overlays/spi1-1cs.dtbo
$ dtmerge base.dtb merged2.dtb /boot/firmware/overlays/spi1-2cs.dtbo
$ dtdiff merged1.dtb merged2.dtb
```

to get:

```
--- /dev/fd/63  2016-05-16 14:18:56.189634286 +0100
+++ /dev/fd/62  2016-05-16 14:18:56.189634286 +0100
@@ -453,7 +453,7 @@

    spi1_cs_pins {
        brcm,function = <0x1>;
-       brcm,pins = <0x12>;
+       brcm,pins = <0x12 0x11>;
        phandle = <0x3e>;
    };

@@ -725,7 +725,7 @@
    #size-cells = <0x0>;
    clocks = <0x13 0x1>;
    compatible = "brcm,bcm2835-aux-spi";
-    cs-gpios = <0xc 0x12 0x1>;
+    cs-gpios = <0xc 0x12 0x1 0xc 0x11 0x1>;
    interrupts = <0x1 0x1d>;
    linux,phandle = <0x30>;
    phandle = <0x30>;

@@ -743,6 +743,16 @@
        spi-max-frequency = <0x7a120>;
        status = "okay";
    };

+
+    spidev@1 {
+        #address-cells = <0x1>;
+        #size-cells = <0x0>;
+        compatible = "spidev";
+        phandle = <0x41>;
+        reg = <0x1>;
+        spi-max-frequency = <0x7a120>;
+        status = "okay";
+    };
};
```

```
spi@7e2150C0 {
```

The **Utils** repo includes another DT utility – `ovmerge`. Unlike `dtmerge`, `ovmerge` combines files and applies overlays in source form. Because the overlay is never compiled, labels are preserved and the result is usually more readable. It also has a number of other tricks, such as the ability to list the order of file inclusion.

Force a specific Device Tree

If you have very specific needs that aren't supported by the default DTBs, or if you just want to experiment with writing your own DTs, you can tell the loader to load an alternate DTB file like this:

```
device_tree=my-pi.dtb
```

Disable Device Tree usage

Device Tree usage is required in Raspberry Pi Linux kernels. For bare metal and other OSs, DT usage can be disabled by adding:

```
device_tree=
```

to `config.txt`.

Shortcuts and syntax variants

The loader understands a few shortcuts:

```
dtparam=i2c_arm=on
dtparam=i2s=on
```

can be shortened to:

```
dtparam=i2c,i2s
```

(`i2c` is an alias of `i2c_arm`, and the `=on` is assumed). It also still accepts the long-form versions: `device_tree_overlay` and `device_tree_param`.

Other DT commands available in `config.txt`

device_tree_address

This is used to override the address where the firmware loads the device tree (not `dt-blob`). By default the firmware will choose a suitable place.

device_tree_end

This sets an (exclusive) limit to the loaded device tree. By default the device tree can grow to the end of usable memory, which is almost certainly what is required.

dtdebug

If non-zero, turn on some extra logging for the firmware's device tree processing.

enable_uart

Enable the **primary/console UART**. If the primary UART is `ttYAMA0`, `enable_uart` defaults to 1 (enabled), otherwise it defaults to 0 (disabled). This stops the core frequency from changing, which would make `ttys0` unusable. As a result, `enable_uart=1` implies `core_freq=250` (unless `force_turbo=1`). In some cases this is a performance hit, so it is off by default.

overlay_prefix

Specifies a subdirectory/prefix from which to load overlays – defaults to `"overlays/"`. Note the trailing `"/"`. If desired you can add something after the final `"/"` to add a prefix to each file, although this is not likely to be needed.

Further ports can be controlled by the DT. For more details see [section 3](#).

Further help

If you've read through this document and have not found the answer to a Device Tree problem, there is help available. The author can usually be found on Raspberry Pi forums, particularly the **Device Tree** forum.

Change the default pin configuration

Custom default pin configurations via user-provided Device Tree blobs has been deprecated.

The recommended route for changing pin configuration is through the device tree. The current instructions are retained for legacy purposes.

Device pins during boot sequence

During the bootup sequence, the GPIO pins go through various actions.

- Power-on – pins default to inputs with default pulls, which are described in the [BCM2835 datasheet](#)
- Setting by the bootrom
- Setting by `bootcode.bin`
- Setting by `dt-blob.bin` (this page)
- Setting by the [GPIO command](#) in `config.txt`
- Additional firmware pins (for example, UARTS)
- Kernel/Device Tree

On a soft reset, the same procedure applies, except for default pulls, which are only applied on a power-on reset.

It may take a few seconds to run through the process. During this time, the GPIO pins may not be in the state expected by attached peripherals (as defined in `dt-blob.bin` or `config.txt`). Since different GPIO pins have different default pulls, you should do **one of the following** for your peripheral:

- Choose a GPIO pin that defaults to pulls as required by the peripheral on reset
- Delay the peripheral's startup until the actions are completed
- Add an appropriate pull-up/pull-down resistor

Provide a custom Device Tree blob

In order to compile a Device Tree source (`.dts`) file into a Device Tree blob (`.dtb`) file, the Device Tree compiler must be installed by running `sudo apt install device-tree-compiler`. The `dtc` command can then be used as follows:

```
$ sudo dtc -I dts -O dtb -o /boot/firmware/dt-blob.bin dt-blob.dts
```

Similarly, a `.dtb` file can be converted back to a `.dts` file, if required.

```
$ dtc -I dtb -O dts -o dt-blob.dts /boot/firmware/dt-blob.bin
```

Sections of the dt-blob

The `dt-blob.bin` is used to configure the binary blob (VideoCore) at boot time. It is not currently used by the Linux kernel. The `dt-blob` can configure all versions of the Raspberry Pi, including the Compute Module, to use the alternative settings. The following sections are valid in the `dt-blob`:

videocore

This section contains all of the VideoCore blob information. All subsequent sections must be enclosed within this section.

pins_*

There are a number of separate `pins_*` sections, based on particular Raspberry Pi models, namely:

- `pins_rev1`: Rev1 pin setup. There are some differences because of the moved I2C pins.

- `pins_rev2`: Rev2 pin setup. This includes the additional codec pins on P5.
- `pins_bplus1`: Raspberry Pi 1 Model B+ rev 1.1, including the full 40pin connector.
- `pins_bplus2`: Raspberry Pi 1 Model B+ rev 1.2, swapping the low-power and lan-run pins.
- `pins_aplus`: Raspberry Pi 1 Model A+, lacking Ethernet.
- `pins_2b1`: Raspberry Pi 2 Model B rev 1.0; controls the SMPS via I2C0.
- `pins_2b2`: Raspberry Pi 2 Model B rev 1.1; controls the SMPS via software I2C on 42 and 43.
- `pins_3b1`: Raspberry Pi 3 Model B rev 1.0
- `pins_3b2`: Raspberry Pi 3 Model B rev 1.2
- `pins_3bplus`: Raspberry Pi 3 Model B+
- `pins_3aplus`: Raspberry Pi 3 Model A+
- `pins_pi0`: Raspberry Pi Zero
- `pins_pi0w`: Raspberry Pi Zero W
- `pins_pi02w`: Raspberry Pi Zero 2 W
- `pins_cm0`: Raspberry Pi Compute Module Zero
- `pins_cm`: Raspberry Pi Compute Module 1. The default for this is the default for the chip, so it is a useful source of information about default pull-ups/pull-downs on the chip.
- `pins_cm3`: Raspberry Pi Compute Module 3
- `pins_cm3plus`: Raspberry Pi Compute Module 3+
- `pins_cm4`: Raspberry Pi Compute Module 4
- `pins_4b`: Raspberry Pi 4 Model B
- `pins_cm4s`: Raspberry Pi Compute Module 4S

Each `pins_*` section can contain `pin_config` and `pin_defines` sections.

`pin_config`

The `pin_config` section is used to configure the individual pins. Each item in this section must be a named pin section, such as `pin@p32`, meaning GPIO32. There is a special section `pin@default`, which contains the default settings for anything not specifically named in the `pin_config` section.

`pin@pinname`

This section can contain any combination of the following items:

- `polarity`
 - `active_high`
 - `active_low`
- `termination`
 - `pull_up`
 - `pull_down`
 - `nopulling`
- `startup_state`
 - `active`

- inactive
- function
 - input
 - output
 - sdcard
 - i2c0
 - i2c1
 - spi
 - spi1
 - spi2
 - smi
 - dpi
 - pcm
 - pwm
 - uart0
 - uart1
 - gp_clk
 - emmc
 - arm_jtag
- drive_strength_mA

The drive strength is used to set a strength for the pins. Please note that you can only specify a single drive strength for the bank. <8> and <16> are valid values.

pin_defines

This section is used to set specific VideoCore functionality to particular pins. This enables the user to move the camera power enable pin to somewhere different, or move the HDMI hotplug position: these are things that Linux does not control. Please refer to the example DTS file below.

Clock configuration

It is possible to change the configuration of the clocks through this interface, although it can be difficult to predict the results! The configuration of the clocking system is very complex. There are five separate PLLs, and each one has its own fixed (or variable, in the case of PLLC) VCO frequency. Each VCO then has a number of different channels which can be set up with a different division of the VCO frequency. Each of the clock destinations can be configured to come from one of the clock channels, although there is a restricted mapping of source to destination, so not all channels can be routed to all clock destinations.

Here are a couple of example configurations that you can use to alter specific clocks. We will add to this resource when requests for clock configurations are made.

```
clock_routing {
    vco@PLLA {    freq = <196608000>; };
    chan@APER {  div  = <4>; };
    clock@GPCLK0 { pll = "PLLA"; chan = "APER"; };
};

clock_setup {
    clock@PWM { freq = <2400000>; };
    clock@GPCLK0 { freq = <12288000>; };
    clock@GPCLK1 { freq = <25000000>; };
};
```


The above will set the PLLA to a source VCO running at 1.96608 GHz (the limits for this VCO are 600 MHz – 2.4 GHz), change the APER channel to /4, and configure GPCLK0 to be sourced from PLLA through APER. This is used to give an audio codec the 12288000Hz it needs to produce the 48000 range of frequencies.

Sample Device Tree source file

The firmware repository contains a [master Raspberry Pi blob](#) from which others are usually derived.

You can view and edit the Raspberry Pi documentation source [on Github](#). Please read our [usage and contributions policy](#) before you make a Pull Request.

Raspberry Pi documentation is copyright © 2012-2026 Raspberry Pi Ltd and is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International](#) (CC BY-SA) licence.

Some content originates from the [eLinux wiki](#), and is licensed under a [Creative Commons Attribution-ShareAlike 3.0 Unported](#) licence.

The terms HDMI, HDMI High-Definition Multimedia Interface, HDMI trade dress and the HDMI Logos are trademarks or registered trademarks of HDMI Licensing Administrator, Inc